A Real-Time Hardware-in-the-Loop Secure-Network Test-Bed Construction for Wind-farms

Áron Dömötör Kohlhéb 5094704

Summary

Power plants continue to be subject to an increasing number of attacks, which increases the need for the development of secure networks, which includes the testing of said networks with increasingly accurate test environments. To test networks it is important to have an accurate representation of delay, having a real-time simulation and incorporating the network as a physical interface produces very realistic delays. Thus this report establishes the attempted construction of a real-time, hardware-in-the-loop test-bed for secure networks. The chosen power plant is a wind farm and the construction of the test-bed is conducted at the lab of Delft Center for Systems and Control (DCSC).

In order to connect the power demand to the wind power generation it is empirical to have a network to connect the turbines of a wind farm. Furthermore, due to wake interactions between turbines, the application of a centralized controller is beneficial for improving overall performance to reduce the wake affects at each turbine. Such a wake field is depicted on Figure 1. Since the focus of the simulation is to test cybersecurity protocols on the communication system, and the simulation needs to run real-time, the fidelity of the models used are relatively low.

The communication network is a target for a attackers, as power production disruption is strong threat. Therefore the central communication network of a wind farm is the subject of the test-bed. Subsequently it is this network that is included as the hardware-in-the-loop. The architecture and hardware used is shown on Figure 2

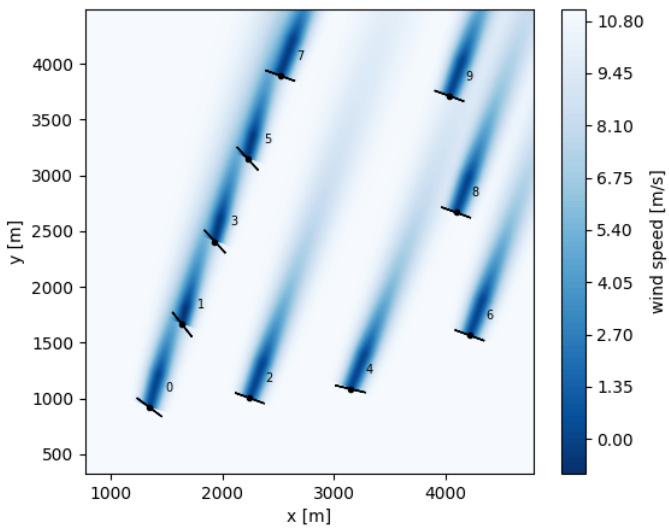


Figure 1: Py-wake: Wake field with wind speed of $11m/s$ and wind direction of 200°

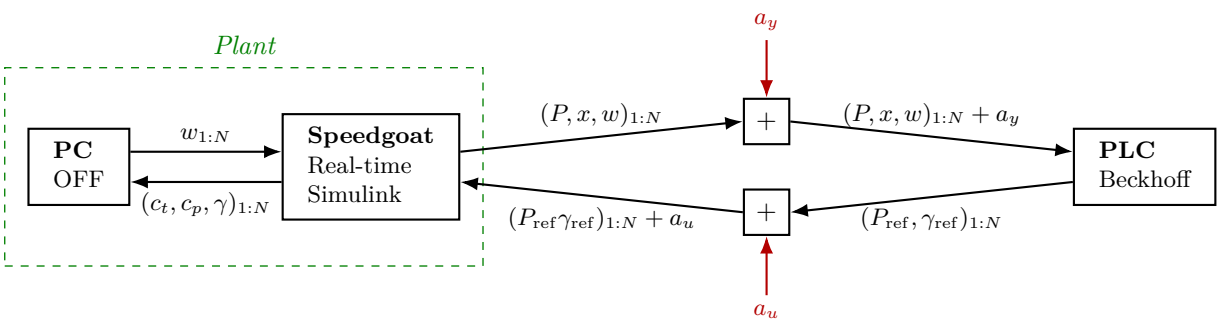


Figure 2: Simulation High-level Architecture with arrowheads

In the context of the network, the plant consists of the turbines, and the wind flow, while the controller is the centralized controller. To represent real life applications at a higher degree of accuracy the controller is implemented in Structured Text (ST) in the Beckhoff environment and runs on a Programmable Logic Controller (PLC). To simulate the Plant real times the turbine simulations run on Speedgoat, which is a Real Time Simulator (RTS), and the wind flow is simulated in a semi-low fidelity wake dynamics framework: OnWaRDS, FLORIDyn and FLORIS (OFF) on a high-performance PC. The information flow is summarized by Table 1 where the notations are described in Table 3. The models and controllers are available at github/arondomotorkohlheb/off_testbed.

The implantation of the models and controllers on the hardware and establishing the communication has been fully explored with the available resources but ultimately every configuration presented at least one issue which is visualized on Figure 3 and is further detailed in the following points.

Computational Part	Software	Hardware	Inputs	Outputs
Wake field simulation	OFF	PC	$(c_t, c_p, \gamma)_{1:N}$	$w_{1:N}$
Turbine model & controller	Simulink	Speedgoat	$(w, P_{\text{ref}}, \gamma_{\text{ref}})_{1:N}$	$(P, x, w, c_t, c_p, \gamma)_{1:N}$
Centralized controller	Structured Text	PLC	$(P, x, w)_{1:N}$	$(P_{\text{ref}}, y_{\text{ref}})_{1:N}$

Table 1: Inputs and outputs for each component in the wind farm test-bed.

Symbol	Definition
c_t	Thrust coefficient of a turbine.
c_p	Power coefficient of a turbine.
γ	Yaw angle (absolute).
w	Wind states.
P	Power generated by a turbine.
P_{ref}	Reference power.
γ_{ref}	Reference absolute yaw angle.
x	Turbine state vector.
$(\cdot)_{1:N}$	Information vertically stacked for all N turbines.
N	Total number of turbines in the wind farm.

Table 2: Definition of variables used in the test-bed architecture.

Issue 1 & Issue 3

The RTS and the PC are connected with an Ethernet connection only. The Ethernet connection serves as the channel to configure the RTS, and Ethernet communication is supported for simulation updates, meaning there are UDP/TCP implementations ready to use for dSpace and Speedgoat, therefore Ethernet connection is the right choice for this communication. It is not advised to build another bus for this communication.

Issue 2

The dSpace hardware available is the MicroAutoBox II model, which requires it's own Ethernet license, which is not available for this project, and support for it's purchase expires at the end of 2026.

Issue 4

`TF6310` licence is missing for PLC to use UDP send and read. It can be purchased to fix the issue.

Issue 5

Since the PLC has Modbus TCP license it was explored as an option, but Speedgoat doesn't have a license to support it.

Issue 6

PLC needs a dedicated hardware interface to work with CAN, which the lab doesn't have.

Issue 7

Both PLC and Speedgoat support EtherCAT but both of them can only function as the master, while in a EtherCAT network there can be only one master. To solve this problem either a microchip chard needs to be added to the Speedgoat that supports functioning as a slave or a bridge can be used that allows both RTS and PLC to send master signals and the bridge converts one of the signals to a slave signal. In both cases there is hardware missing.

Issue 8

Speedgoat only has 4 output analogue channels while the number of values to send is at least 7 per turbine, and for a wind farm to make sense it should include at least 10 turbines, meaning that there are 70 values to send. The problem is also present in the PLC to RTS direction.

Issue 9

I/O channels can be used to make a decoding and encoding of the signals, which is outside of the scope of this project as it is very close to hardware and comes with another set of problems and frameworks to solve on a lower level.

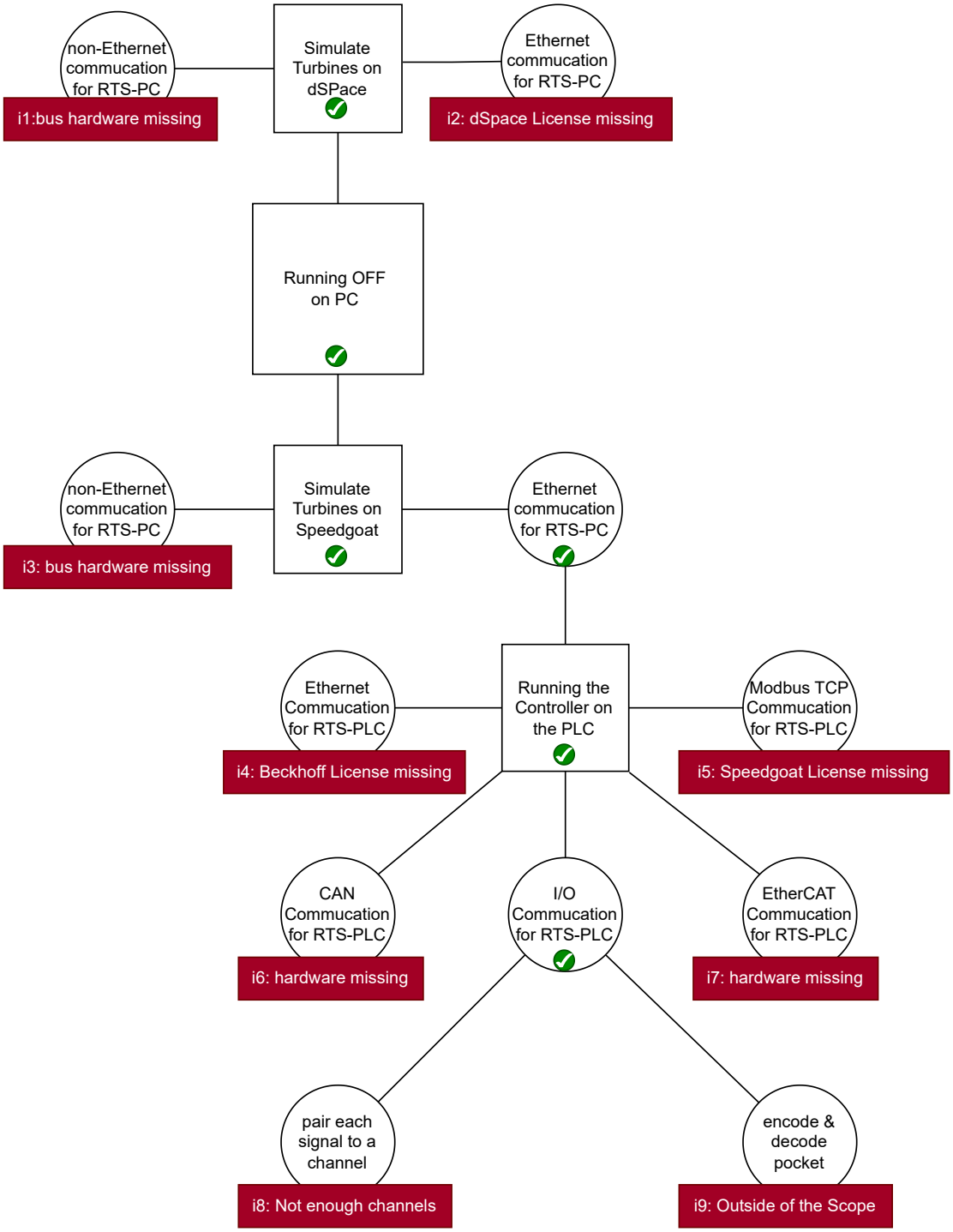


Figure 3: Hardware Implementation Option Tree

Contents

1 Introduction **5**
1.1 Wind Turbines . . . 5

2 Test-bed Goals **6**

3 Wind Turbine Modelling & Control **8**
3.1 Modelling . . . 8

4 Local Controller **9**
4.1 PI Controller . . . 11
4.2 β Controller . . . 11

5 Single Turbine Results **14**

6 Wake-Field Simulation **15**
6.1 Overview of OFF . . . 15
6.2 Overview of FLORIS . . . 16
6.3 Farm Layout . . . 17
6.4 Greedy Farm . . . 18
6.5 Uniform Power Tracking Farm Results . . . 19

7 Centralized Control **20**
7.1 Power Distribution . . . 20
7.2 Yaw Steering . . . 21
7.3 Results . . . 22

8 Implementation on Hardware **27**
8.1 Intended Architecture . . . 27
8.2 Implemented Architecture . . . 27
8.3 Using Speedgoat . . . 27
8.4 RTS - PC Connection . . . 28
8.5 Configuring the PLC . . . 29
8.6 Current Issues . . . 30

9 User Manual **32**
9.1 Quick Start . . . 32
9.2 Customizing the Wind Farm and Wind Conditions . . . 32
9.3 Customizing Communication . . . 33
9.4 Recommended Solutions to the Standing Problem . . . 33
9.5 Not Recommanded Solutions . . . 34

A Changes in OFF **35**
A.1 off_interface updates . . . 35
A.2 Turbine module changes . . . 35
A.3 OFF interface and wake model changes . . . 35

B Changes in Floris **36**

C x_{ref} Mapping **36**

D Yaw Optimization with PyWake **37**

1 Introduction

In this report, after a a brief introduction on wind turbines, first the requirements are established for the testbed in section 2. Then model and local controller of a single turbine are described in section 3. Then the wake-field computations are described in section 6, followed by the centralized controller in section 7. Finally the implementation on the system of hardware is described in section 8. The models and controllers are available at github/arondomotorkohlheb/off_testbed.

1.1 Wind Turbines

The aerodynamic behaviour of a wind farm is governed by the interaction between the atmospheric wind field and the individual turbines, whose performance is primarily determined by the effective wind speed at the rotor plane and the turbine's aerodynamic characteristics. These characteristics are commonly represented by the power coefficient c_p and the thrust coefficient c_t, which describe the conversion of the kinetic energy of the flow into power by a torque on the rotor and a thrust force respectively.

The aerodynamic power converted into rotation of the rotor is given by the left side of Equation 1.

$$P_{ae}(V,\lambda,\beta) = \frac{1}{2}\rho A V^3 \cos^3(\gamma_{rel})\, c_p(\lambda,\beta)\, I \qquad F_t(V,\lambda,\beta) \quad = \frac{1}{2}\rho A V^2 \cos^3(\gamma_{rel})\, c_t(\lambda,\beta)\, I \tag{1}$$

where ρ is the air density, A is the rotor swept area, V denotes the incoming wind speed, λ is the tip-speed ratio, β is the blade pitch angle, and γ_{rel} is the relative yaw misalignment between the rotor axis and the wind direction. The factor $\cos^3(\gamma_{rel})$ captures the reduction in effective wind speed due to yaw, while the turbulence intensity I which is a coefficient that represents energy reduction in the flow due to turbulance.

Similarly, the aerodynamic thrust force acting on the turbine is expressed as The thrust coefficient c_t as it is closely related to boundary layer separation, which strongly related to wake generation. Increasing the power extracted from the wind, meaning c_p, also increases the wake generation (c_t), meaning that the downstream turbines maximum power generation is reduced.

The mechanical power extracted from the wind is converted into electrical power through the generator dynamics. The generated power is given by

$$P_{gen} = \tau_{gen}\, \omega\, \mu,$$

where τ_{gen} is the generator torque, ω is the rotational speed of the rotor, and μ represents the overall drivetrain and generator efficiency. This relation links the aerodynamic subsystem to the electrical output of the turbine.

In a wind farm consisting of N turbines, the aerodynamic coupling between turbines arises from wake interactions. The thrust forces $T_{1:N}$ determine the strength and spatial extent of the wakes, while the relative yaw angles $\gamma_{rel,1:N}$ influence their deflection. As a result, the effective wind speed experienced by each turbine becomes a non-linear function of the entire farm configuration. This coupling fundamentally limits the performance of greedy operation and motivates coordinated control strategies that account for both local power extraction and global wake effects across the wind farm.

The notation is summarized in Table 3 with the most important parameters visualized on Figure 4

Symbol	Definition
c_t	Thrust coefficient of a turbine.
c_p	Power coefficient of a turbine.
γ	Yaw angle (absolute).
λ	Tip-speed ratio.
V	Effective wind speed at the turbine rotor.
γ_{wind}	Wind direction at the turbine rotor.
w	Wind states, which is the stacking of V and γ_{wind}.
P_{ae}	Power captured in rotor rotation by a turbine.
P_{gen}	Power generated by a turbine.
F_t	Thrust force experienced by a turbine.
x	Turbine states, which is the stacking of pitch, yaw, and rotor angular velocity.
I	Turbulence intensity, a coefficient that represents the reduction in energy in the flow.
A	Rotor area.

Table 3: Definition of variables used in the test-bed architecture.

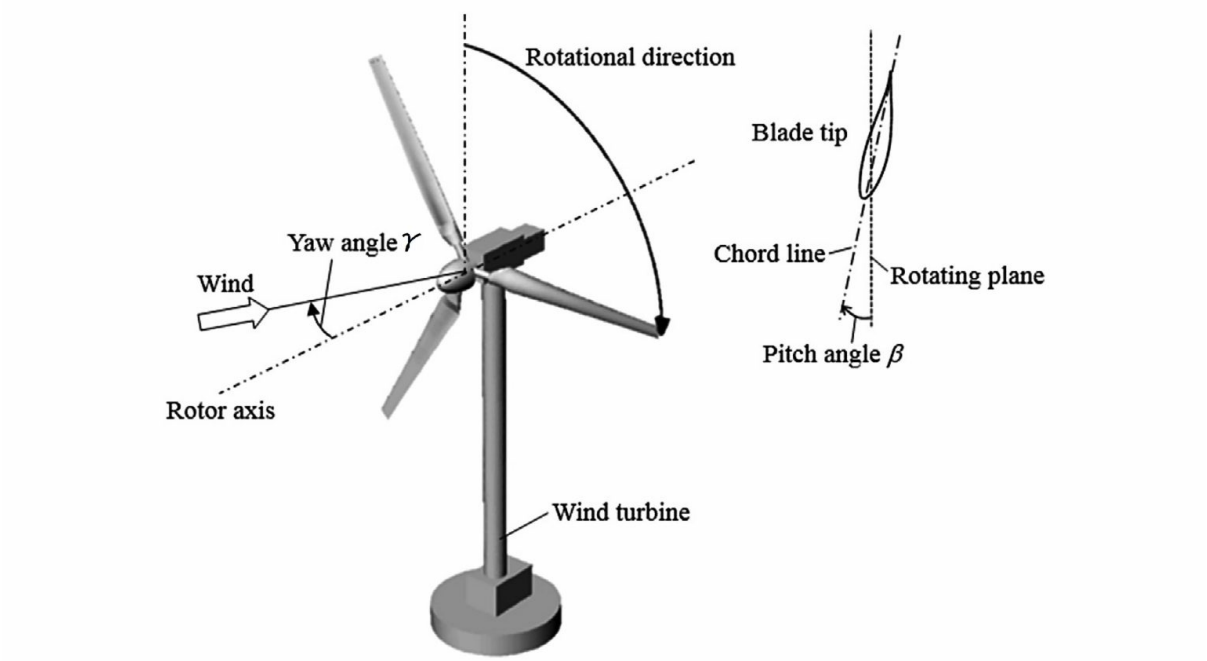


Figure 4: Turbine Parameters [Buccafusca and Beck, 2021]

2 Test-bed Goals

In this chapter the necessary conditions on the implementation to construct the test-bed are described in the form of a high-level requirements. The test-bed can be built in multiple ways, but to include the network as the hardware-in-the-loop, and to make the test-bed to run real-time the following requirements have to be met:

Req-Plant-01 The plant (wind farm) shall be simulated real-time

The main advantage of making the test-bed and including the hardware in the loop is to very accurately model the delay present in the network. For the delay to have consistent effects on the dynamics the simulation needs to run in real time. Real-time simulations are widely used for test-beds, therefore Real Time Simulators (RTS) have been developed with special hardware that enforces real-time on the computations. Therefore the first condition is to simulate the plant with an RTS. In order to allow the usage of already established wake-field simulators that run on python the plant is simulated on the combination of RTS and a PC.

Req-Controller-01 The centralized controller shall imitate industrial implementation

In order to properly test the effects of delay for industrial application the controller should resemble an industrial controller. Programmable Logic Controllers (PLCs) are custom made and widely used hardware to run controllers for large span of time without stopping, therefore a PLC is chosen to run the controller in the test-bed.

Req-Bus-01 The communication has to resemble real-time communication

The communication protocol has the largest effect on delays. For example having a crowded traffic on the bus adds significant delay to the communication, and may even disrupt some packets. In order to test for industrial applications optimally the protocol also is a real-time protocol, such as EtherCAT or CAN. These protocols distribute the usage of the network among the nodes to send signals with a time-based schedule. Since in the case of this test-bed there is only communication between two nodes, real-time protocols can be emulated with a non-real-time protocol, as long as the packets arrive without additional delay caused by traffic.

The requirements are also presented as a layout for the required architecture presented on Figure 5. The possible versions of the architecture are further detailed in section 8 after the members are described in more detail as the wind turbine section 3, wake-field section 6 and centralized controller section 7.

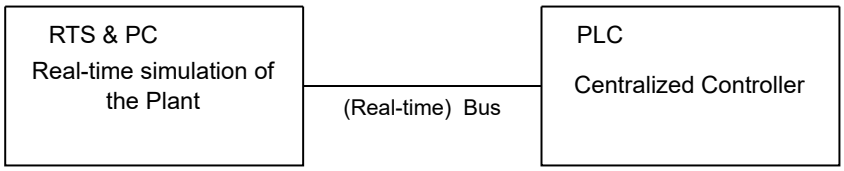


Figure 5: Required High-level Architecture

3 Wind Turbine Modelling & Control

The first model discussed is the model of a single turbine. The model is built with the simplifying assumption of aerodynamic forces being based on c_p and c_t in subsection 3.1. Furthermore the local controller is also developed such that the turbine tracks power production while aiming to minimize the generated wake in section 4.

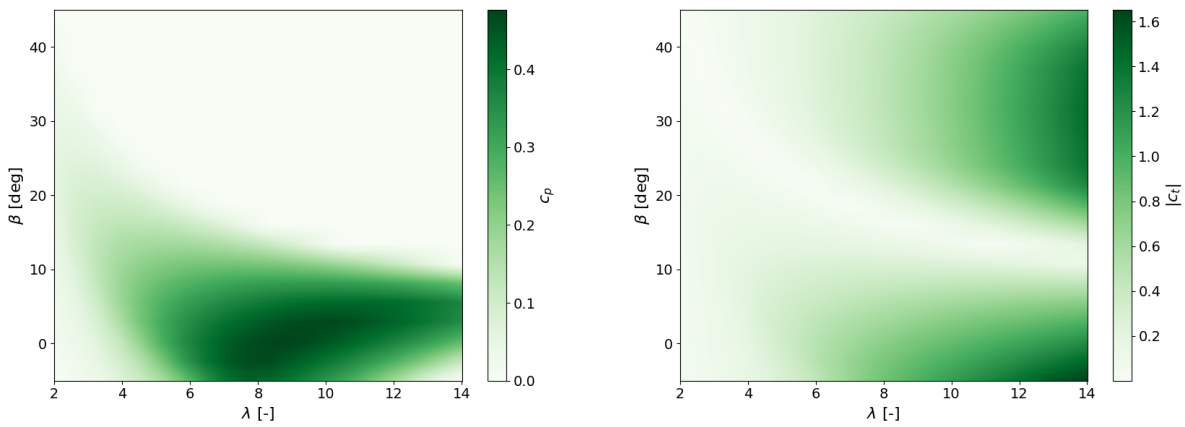
3.1 Modelling

To build the wind-farm model the turbine IEA-22-280-RWT, which is a 22-MW turbine with accessible data to support research. The parameters used in this project are taken from Zahle et al., 2024 and are shown on Table 4.

Parameter	Notation	Value
Cut-in wind speed [m/s]	$V_{\text{cut-in}}$	3.0
Rated wind speed [m/s]	V_{rated}	11.0
Cut-out wind speed [m/s]	$V_{\text{cut-out}}$	25.0
Rotor radius [m]	r_{rotor}	142
Second moment of inertia of one blade [kg m^2]	I_{rotor}	$2.052 \cdot 10^8$

Table 4: Parameters of IEA-22-280-RWT

Furthermore the c_p and c_t values of the turbine are estimated based on the pitch (β) and tip speed ratio (λ). In this approach the $c_p(\beta, \lambda)$ and $c_t(\beta, \lambda)$ functions are constructed as the linear interpolation of the tables, represented as heat maps on Figure 6. The plots are preprocessed such that for the c_p plot only the non-negative values are shown as negative values are avoided. In case of the c_t plot the absolute value is shown as the higher the absolute value of c_t the higher the generated wake.



(a) c_p heat map (only non-negative

(b) absolute value of c_t heat map

Figure 6: Aerodynamic performance maps: power and thrust coefficients

The graph shows that higher pitch angles β are associated with a lower value of c_t and thus less turbulence. This trend is utilized later in the local controller design. The aerodynamic forces are modelled with the standard approach Equation 2 and Equation 3.

$$P_{ae}(V, \lambda, \beta) = \frac{1}{2}\rho A V^3 cos(\gamma_{rel})^3 c_p(\lambda, \beta) I \tag{2}$$

$$F_t(V, \lambda, \beta) = \frac{1}{2}\rho A V^3 cos(\gamma_{rel})^2 c_t(\lambda, \beta) I \tag{3}$$

where I is the turbulence intensity. The power generated is expressed as

$$P_{gen} = \tau_{gen}\omega\mu \tag{4}$$

where μ is the efficiency of the generator.

The turbine is modelled with the states x. The update equations for the pitch and the yaw are modelled as a switching systems directly driven by the control inputs u. Furthermore the wind is expressed as the stacking of its speed and direction in w.

$$x^\top = \begin{pmatrix} \beta & \gamma & \omega \end{pmatrix} \tag{5}$$

$$u^\top = \begin{pmatrix} u_\beta & u_\gamma & \tau_{gen} \end{pmatrix} \tag{6}$$

$$w^\top = \begin{pmatrix} V & \gamma_{wind} & I \end{pmatrix} \tag{7}$$

$$\dot{\beta} = \begin{cases} c_{\dot{\beta}}, & \text{if } u_\beta = 1 \\ -c_{\dot{\beta}}, & \text{if } u_\beta = -1 \\ 0, & \text{if } u_{\dot{\beta}} = 0 \end{cases} \tag{8}$$

$$\dot{\gamma} = \begin{cases} c_{\dot{\gamma}}, & \text{if } u_\gamma = 1 \\ -c_{\dot{\gamma}}, & \text{if } u_\gamma = -1 \\ 0, & \text{if } u_{\dot{\gamma}} = 0 \end{cases} \tag{9}$$

$$\dot{\omega} = \frac{\tau_{ae}(w, x) - \tau_{gen}}{J_t} \tag{10}$$

The states are updated with assumed zero order hold on the variable in-between time steps as

$$x_{k+1} = x_k + T_s \dot{x} \tag{11}$$

4 Local Controller

The local controller of the turbine sets the pitch and the generator torque such that the turbine produces the requested power under the requested yaw. To track power translates to tracking c_p via Equation 12.

$$c_{p.ref} = max\left(\frac{P_{ref}}{\frac{1}{2}\rho A V^3 cos(\gamma_{rel})^3 I}, c_{p.max}\right) \tag{12}$$

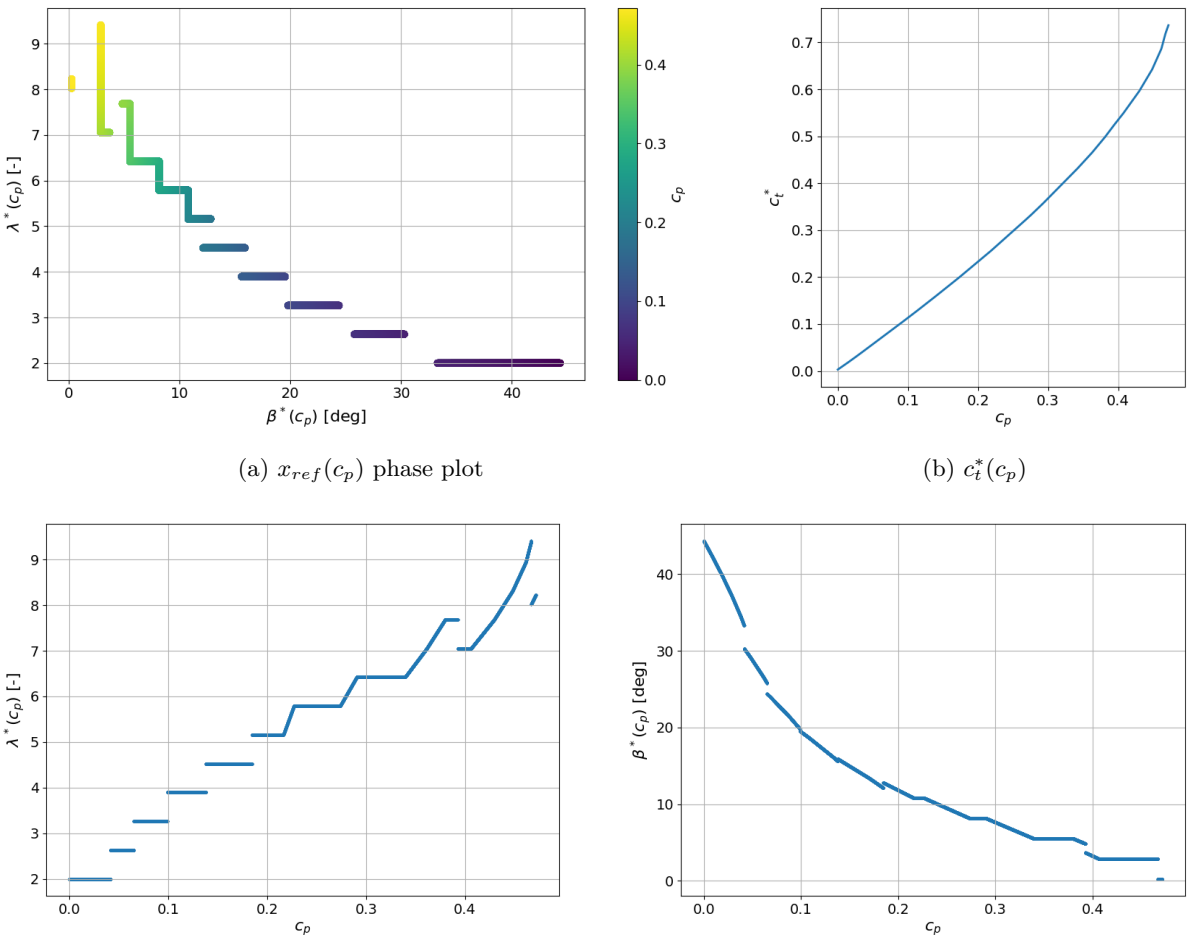
As Figure 6a shows the solution to $c_p(\beta, \lambda) = c_{p.ref}$ does not have a unique solution. Since lower c_t value is related to lower turbulence generation the problem is formulated as Equation 13.

$$\begin{aligned} x_{ref} &= arg \min_x c_t(x) \\ &s.t. \\ c_p(x) &= c_{p.ref} \end{aligned} \tag{13}$$

The problem is solved numerically. Since both functions are given as linear interpolations their structure is used to look for the solution. On the bounded plane finding the intersection with a constant gives an empty set, one point, or a straight line between two points. Since $c_p(x)$ and $c_t(x)$ are given in tiles of bounded planes this property is exploited for each tile separately. If there is a solution to find x_{ref} the following property is used: the minimum or maximum value of any bounded plane will lie on its boundary. Resulting in the insight that all optimal values for x_{ref} will lie on the intersection between the adjacent points defined by the tables represented in Figure 6.

Therefore to solve Equation 13 for each x on the boundaries such that $c_p(x) = c_{p.ref}$ are found and by computing the c_t value for each the one that has the lowest absolute value of c_t is the optimal one. This process is repeated for values of $c_{p.ref}$ on segment $(0, c_{p.max})$ which is uniformly distributed to N_{res} points. For $N_{res} = 1e5$ the results are shown as both a phase plot, $c_t(c_{p.ref})$, $\beta_{ref}(c_{p.ref})$ and $\lambda_{ref}(c_{p.ref})$ on Figure 7. The results are used as a one-dimensional look-up table meaning that $x_{ref}(c_{p.ref})$ is computed off-line .

In addition to this process. the online finding of x_{ref} includes a limitation that due to having a switch to change β only discrete values of β are achievable, hence the x_{ref} is found such that β_{ref} is an achievable value. The set of achievable values of β is denoted as $\mathcal{B}$.



(a) $x_{ref}(c_p)$ phase plot

(b) $c_t^*(c_p)$

(c) $\lambda^*(c_p)$

(d) $\beta^*(c_p)$

Figure 7: Mapping c_p to optimal states, and corresponding c_t.

4.1 PI Controller

To track x_{ref} first the most simple controller is established, assuming that the dynamics is decoupled, therefore *omega* is steered with the control input τ_{gen}, β is steered with the control input u_β, and γ is steered with u_γ. The switching system for β and γ are presented in Equation 14 and Equation 15 respectively.τ_{gen} is defined with the PI controller with gains presented in Table 6. The information flow is visualized by Figure 8, where each non-externally defined block is referenced to an equation or figure.

$$u_\beta = \begin{cases} 0 & \text{if } |\beta_{target} - \beta| < \frac{1}{2} c_{\dot{\beta}} T_s \\ \begin{cases} 1 & \text{if } \beta_{target} > \beta \\ -1 & \text{if } \beta_{target} < \beta \end{cases} & otherwise \end{cases} \tag{14}$$

$$u_\gamma = \begin{cases} 0 & \text{if } |\gamma_{\text{target}} - \gamma| < \frac{1}{2} c_{\dot{\gamma}} T_s \\ \begin{cases} 1 & \text{if } \gamma_{\text{target}} > \gamma \\ -1 & \text{if } \gamma_{\text{target}} < \gamma \end{cases} & otherwise \end{cases} \tag{15}$$

$$\tau_{gen}(k) = -J_t \left(P(\omega_{ref} - \omega(k)) + I \sum_{i=0}^{k} (\omega_{ref} - \omega(i)) \right) \tag{16}$$

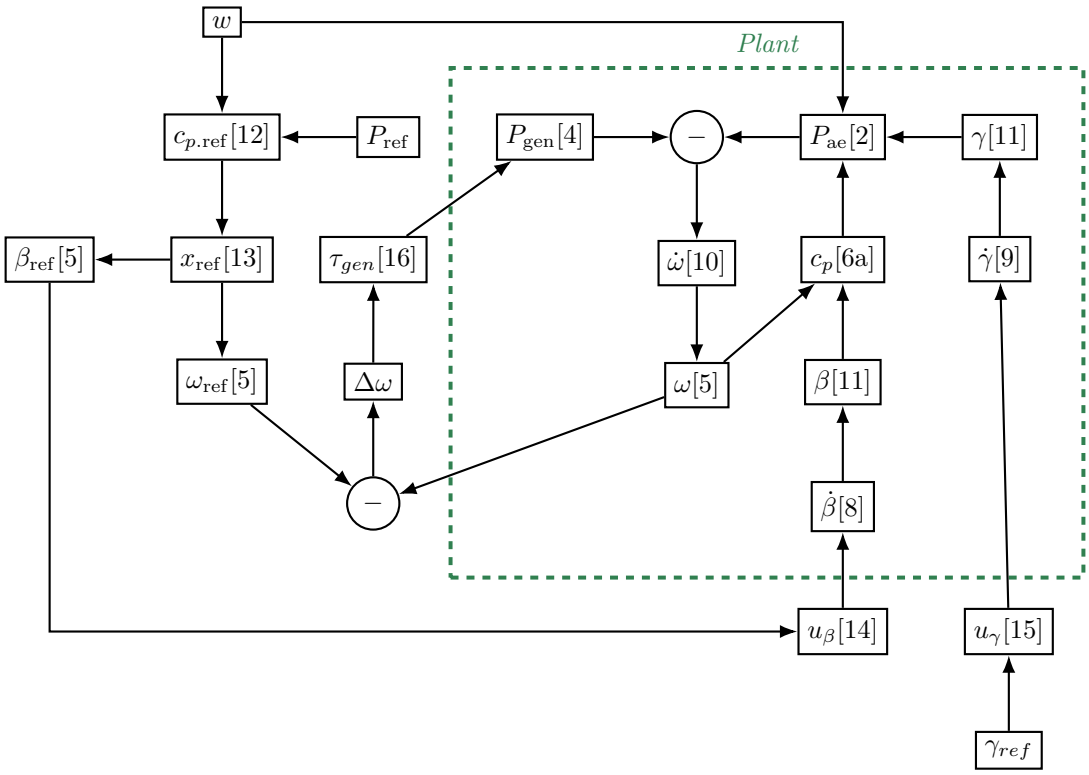


Figure 8: $\beta Controller Information Flow$

4.2 β Controller

Secondly approximating the optimal controller is explored. The optimal controller is defined as the summation of generating the maximal power and minimizing the wake generation while steering x to x_{ref} as x_{ref} is the optimal state to produce P_{ref} power under wind conditions w. It is assumed that while the transition to x_{ref} by $u(t)$ happens, the wind stays constant.In Equation 17 q stands for the weight of the wake generation with respect to the power tracking.

$$u^\star(t) = \arg\min_u \frac{1}{T}\int_0^T (P_{gen}(t) - P_{ref})^2 + q \cdot c_t(t, x(t), w(t))^2 dt \tag{17}$$

$$s.t \int_0^T \dot{x}(u(t), t, w(t))dt = x_{ref} - x_0 \tag{18}$$

To solve this problem the differential equation Equation 19 needs to be solved, but this type of differential equation doesn't have an analytic solution and solving it numerically with a terminal constraint in an optimization loop to find $u^\star(t)$ is not feasible due to how computational expensive it is.

$$\dot{\omega} \cdot J_t = \frac{P_{ae}(w, x)}{\omega} - \tau_{gen} \tag{19}$$

The optimal controller is approximated with a procedure that translates the tracking to target values of c_p and thus β via a P controller, and assumptions that rely on the slow nature of the rate of change in x due to the large inertia J_t. Since τ_{gen} is both a control input and is directly linked to the output to track, two different controllers are defined. Controller a only uses β as a control input while τ_{gen} is set to track P_{ref}, and controller b uses both β and τ_{gen} to reach x_{ref}. Controller a is used when it is possible to track the power perfectly while steering x to x_{ref}, while controller b in all other cases.

The controller is approached with the following assumption: ω doesn't change significantly and that time β takes to change to its target value is insignificant. The controllers assume full information on the states and the wind conditions, which can be further developed in the future to achieve a higher level of realism, but for the construction of the test bed it is sufficient to make this assumption.

Both controllers have the same logic. Via a proportional gain, the difference between ω_{ref} and ω drive the process and is noted as $\dot{\omega}_{target}$. By using the model the target angular acceleration is translated to the control inputs via the following steps

$$\dot{\omega}_{target} = p(\omega_{ref} - \omega) \tag{20}$$

$$\tau_{aero.target} = \dot{\omega}_{target} J_t + \frac{P_{ref}}{\omega\mu} \tag{21}$$

$$c_{p.target} = \frac{\tau_{aero.target}\omega}{\rho A V^3 I cos(\gamma_{rel})^3} \tag{22}$$

$$\tag{23}$$

then β_{target} is found such that with the current ω and thus λ it produces at least $c_{p.target}$

$$\beta_{target} = \arg\min_{\beta \in \mathcal{B}} (c_p(\beta, \lambda) - c_{p.target})^2 \tag{24}$$

The control input u_β is given as the switching system to track β_{target}.

$$u_\beta = \begin{cases} 0 & \text{if } |\beta_{target} - \beta| < \frac{1}{2} c_{\dot{\beta}} T_s \\ \begin{cases} 1 & \text{if } \beta_{target} > \beta \\ -1 & \text{if } \beta_{target} < \beta \end{cases} & otherwise \end{cases} \tag{25}$$

The choice between the two subprocesses takes place at this stage of the control loop. If $\omega_{ref} \leq \omega$ then always controller a is chosen, since then it is always possible to slow down ω by reducing c_p while perfectly tracking P_{ref}. If $\omega_{ref} \geq \omega$ then the following test is used: is c_p achievable large enough that while perfectly tracking P_{ref} ω will increase.

$$c_p(\beta_{target}, \lambda) > \frac{P_{ref}}{\rho A V^3 I cos(\gamma_{rel})^3 \mu} \tag{26}$$

If Equation 26 passes then controller a is used and τ_{gen} is given as

$$\tau_{gen} = \begin{cases} \frac{P_{ref}}{\omega\mu} & if\ \omega \geq 0 \\ 0 & otherwise \end{cases} \tag{27}$$

If Equation 26 doesn't pass then controller b is used and τ_{gen} is altered such that $\dot{\omega}_{target}$ is met.

$$\tau_{gen} = \tau_{ae}(w, x) - \dot{\omega} J_t \tag{28}$$

The information flow is visualized by Figure 9, where each non-externally defined block is referenced to an equation or figure.

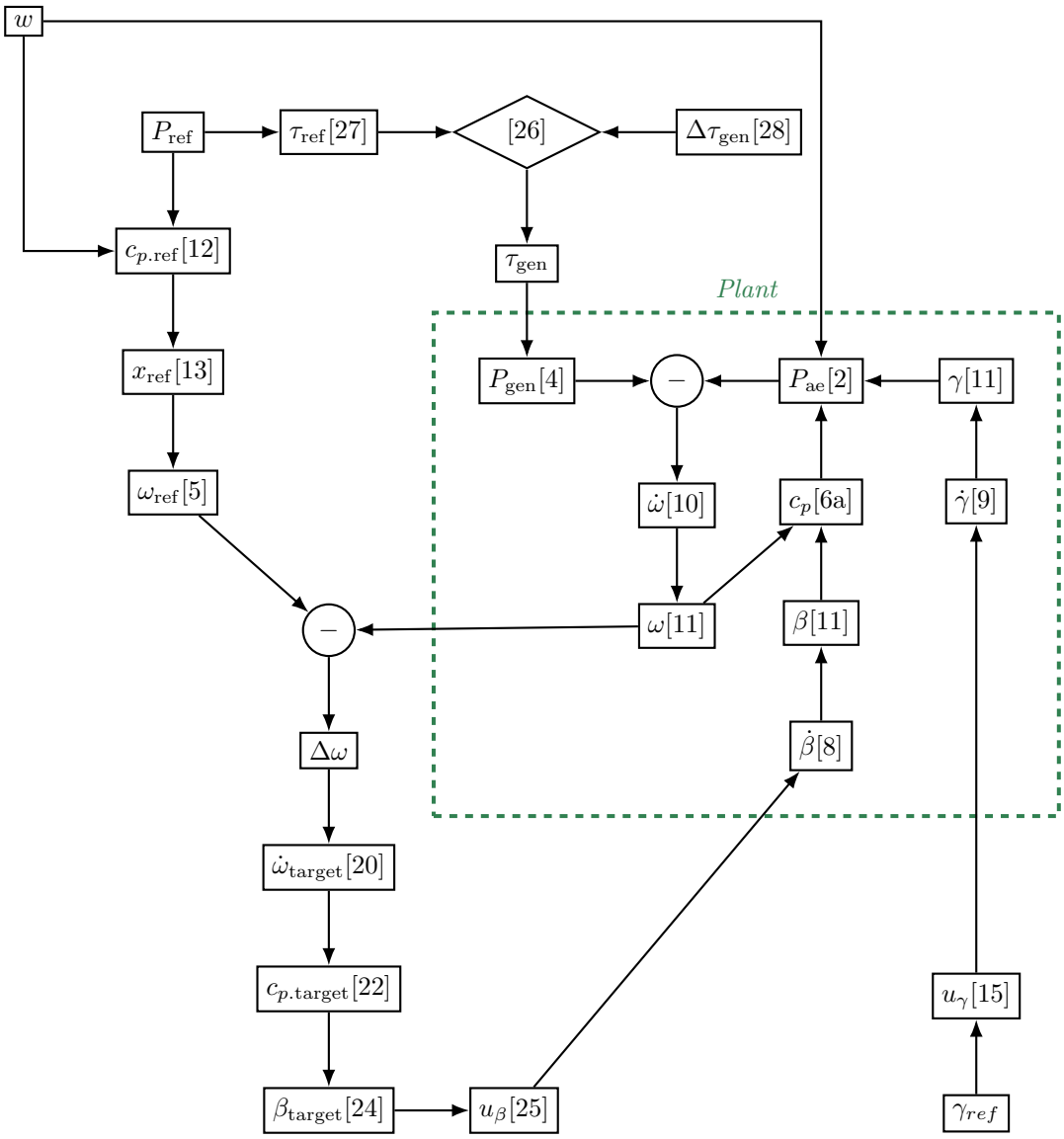


Figure 9: β Controller Information Flow

5 Single Turbine Results

The results shows are generated with $T_s = 0.1$. $V = 11$, $I = 0.06$. Both controllers are tuned for two cases. The tuned values used are displayed on Table 6. The performance is displayed on Figure 10. In the following sections of this report the fast version of the β local controller is used as it is performing significantly better.

Case	**Description**
Case 1	Track power generation as fast as possible
Case 2	Track power generation as fast as possible while as much power is generated as possible

Table 5: Definition of control cases.

Case	**Type**	**sign**$(\Delta\omega)$	**P**	**I**
Case 1	β	{+}	0.5	–
Case 1	β	{-}	0.05	–
Case 1	PI	{+,-}	-1	-0.01
Case 2	β	{+,-}	0.01	–
Case 2	PI	{+,-}	-0.01	-0.001

Table 6: Local Controller Gains

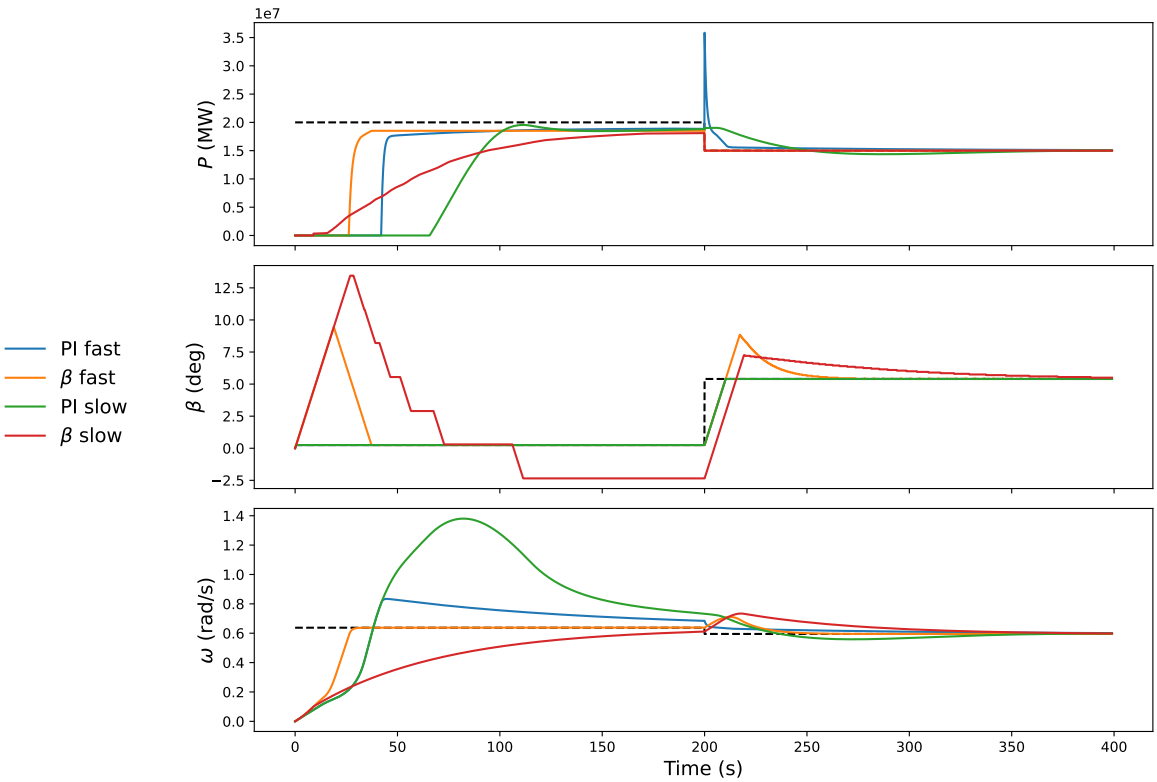


Figure 10: Single Turbine Performance

6 Wake-Field Simulation

In order to alter both OFF and Floris to suit the needs of the testbed the main flow of information and operations has to be understood. Since OFF lacks a non-AI generated documentation the necessary extent of its logic is documented in this report in order to preserve the found structure and to aid the reproducibility, and usability of the altered code for future purposes.

6.1 Overview of OFF

First the main methods and their structure is shown Table 7, then the main logic is described.

→ *off_interface.init_simulation_by_path()*
  → *while latest pocket not received: UDP listen*
  → *run_one_step()*
    → off interface
      → off
        → *get_measurements_reduced*
          → *fmodel.set()*
          → *fmodel.run()*
        → *ambient_corrector.update()*
          → *iterate_states_and_keep()*
          → *propagate_ops()*
  → Sends Simulink input to Speedgoat with UDP
→ *Close socket and Save data*

Table 7: OFF Overview

Init and setting up OFF

The OFF simulation is initialized by inputing the *yaml* configuration file into *off_interface.init_simulation_by_path()*. This configuration file describes the duration and time step of the simulation, the global wind conditions, as well as the geometric turbine parameters such as height and rotor diameter, and turbine placement as layout x,y,z. The turbine model is usually defined by a specifying a turbine type, which is described by c_p and c_t tables, but since OFF is not used to model the turbines in this testbed, a new empty turbine type is created: turbine mask.

Communication

To communicate with Speedgoat via UDPs the following logic is applied: First the python script is started and is waiting to receive a pocket in the Speedgoat to OFF port. Once it reads a pocket it will read as many as it can so it reads the most recent one. When the stack of incoming pockets is empty the simulation part of the script executes. Once the flow field computation is over it sends the new flow field data to Speedgoat.

Wrapping OFF

OFF uses FLORIS to generate the wake and FLORIDYN to propagate them. The flow field simulation is wrapped in *off2Speedgoat* but it passes many layers before the main part of the flow field computation takes place. OFF is interacted with an *off_interface* object, which creates an OFF object. In OFF the flow field is described by a finite number of measurement points (also configured in the configuration file). In addition to that the turbines are described by turbine states and ambient states, ambient states being the wind conditions at each observation point. OFF propagates the information by updating the states

and by moving the measurement points with the flow field. The wake generation is done by FLORIS. To interface with FLORIS the wind farm is set with *fmodel.set()* and is solved with *fmodel.run()*. The new flow field values are loaded into the states via updating the ambient states.

6.2 Overview of FLORIS

Via `off`, `fmodel.set()` is called, which updates the `fmodel.core` object through a `_reinitialization()` and `set_operation()`. The obtained relative yaw angles (γ_{rel}) and thrust coefficients (c_t) are passed at this stage. Since turbine dynamics may result in updated turbine positions, `fmodel.set()` takes the incoming wind conditions, layout, yaw angles, and thrust coefficients. Inside this function, `core.thrust_coefficients` is updated. When the wake-generation computation is executed, c_t is passed explicitly rather than being recomputed.

In the original OFF-floris interaction only the layout and yaw is passed down to floris, which computes power and c_t on it's own. This is the reason that one of the limitations of OFF is that new turbine models, including c_t and c_p tables, have to be implemented in both OFF and Floris turbine library. The re-computation of these values would make the model much worse, as floris assumes greedy power production, therefore reducing the power required from each turbine wouldn't reduce the wakes generated, making the model inconsistent. In order to overcome c_t values has been as an additional property, and the c_t computations inside floris have been replaced with returning this value. This change however influenced how floris works, it takes longer to converge leads to a longer time for the wake-field to converge.

In addition to that the following behaviour is observed with floris: having no relative yaw angle causes the downstream wind to have no significant amount of added turbulence, while there is a significant amount of reduction in velocity. On the other hand having a non-zero relative yaw causes a significant raise in added turbulence, while the velocity reduction is reduced. To study the realism of this behaviour is outside of the scope of this report, thus it remains a point of improvement to increase the accuracy of the wake generation.

6.3 Farm Layout

Even though the test-bed allows for the use of a configurable farm layout, in this report only one farm layout is used for consisteny, which is displayed on Figure 11. In the subsequent plots the turbines are linked to the indexes and colour displayed on this graph.

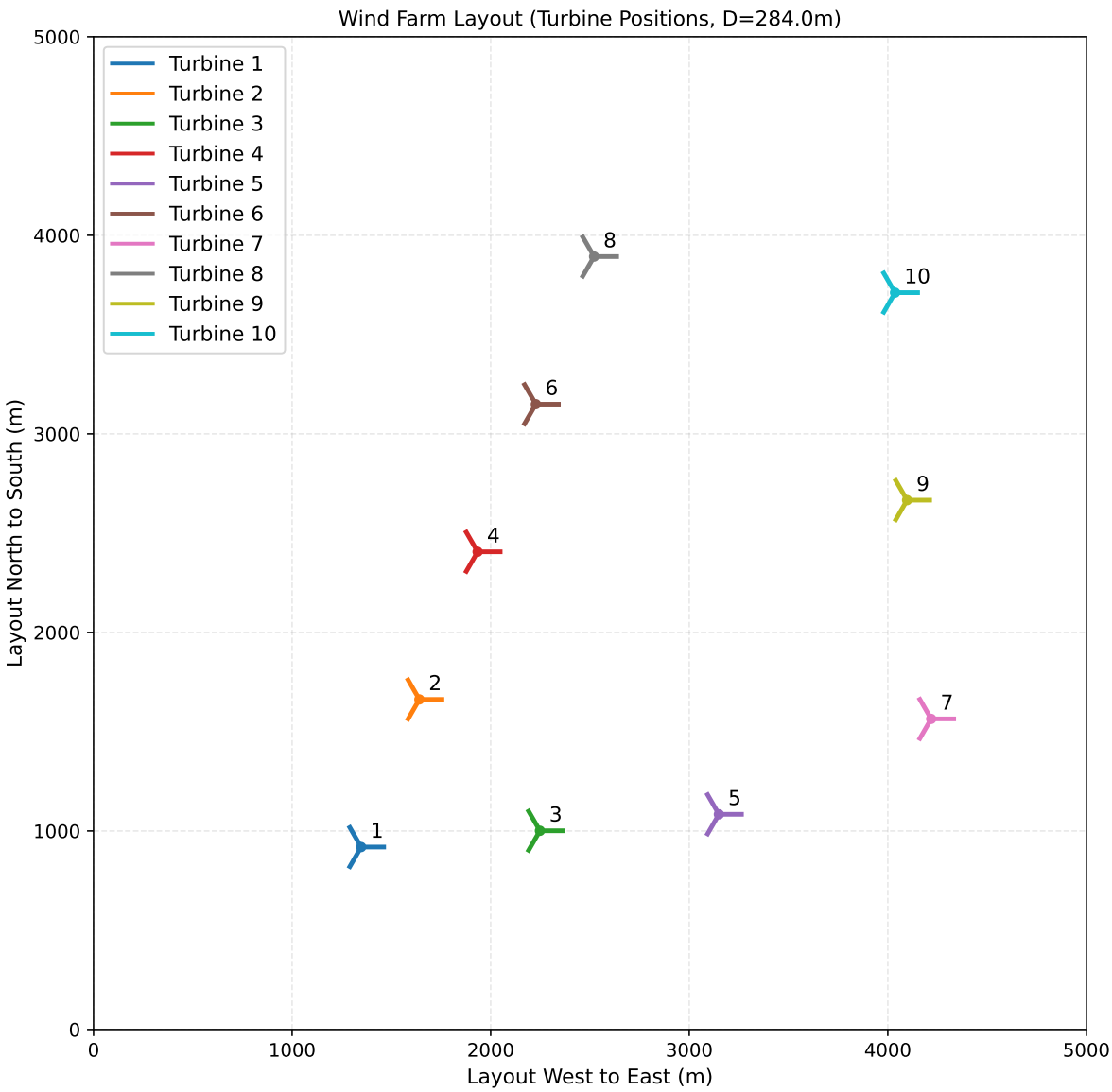


Figure 11: Turbine Layout

6.4 Greedy Farm

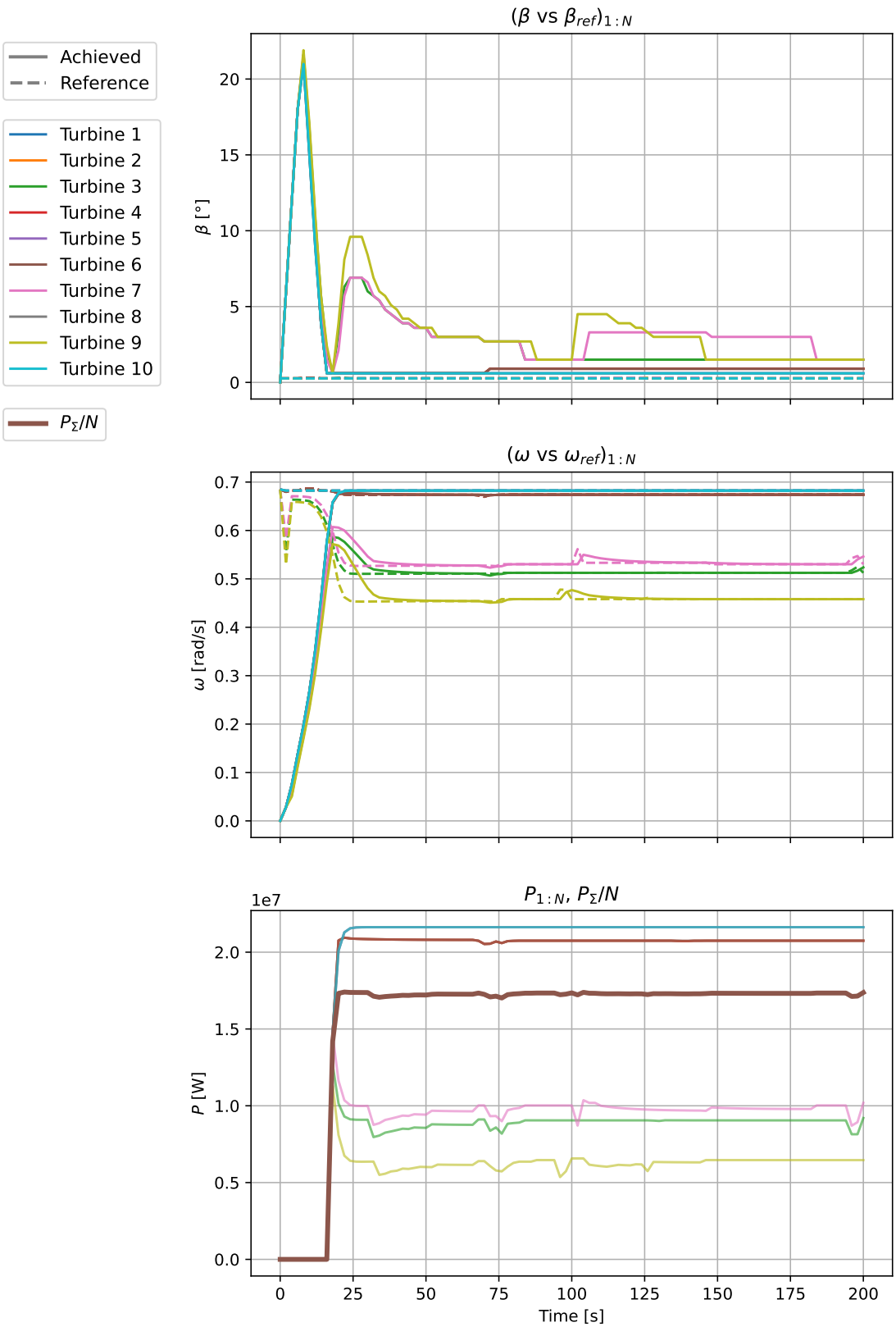


Figure 12: Locally Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 0°, 0.06)$

6.5 Uniform Power Tracking Farm Results

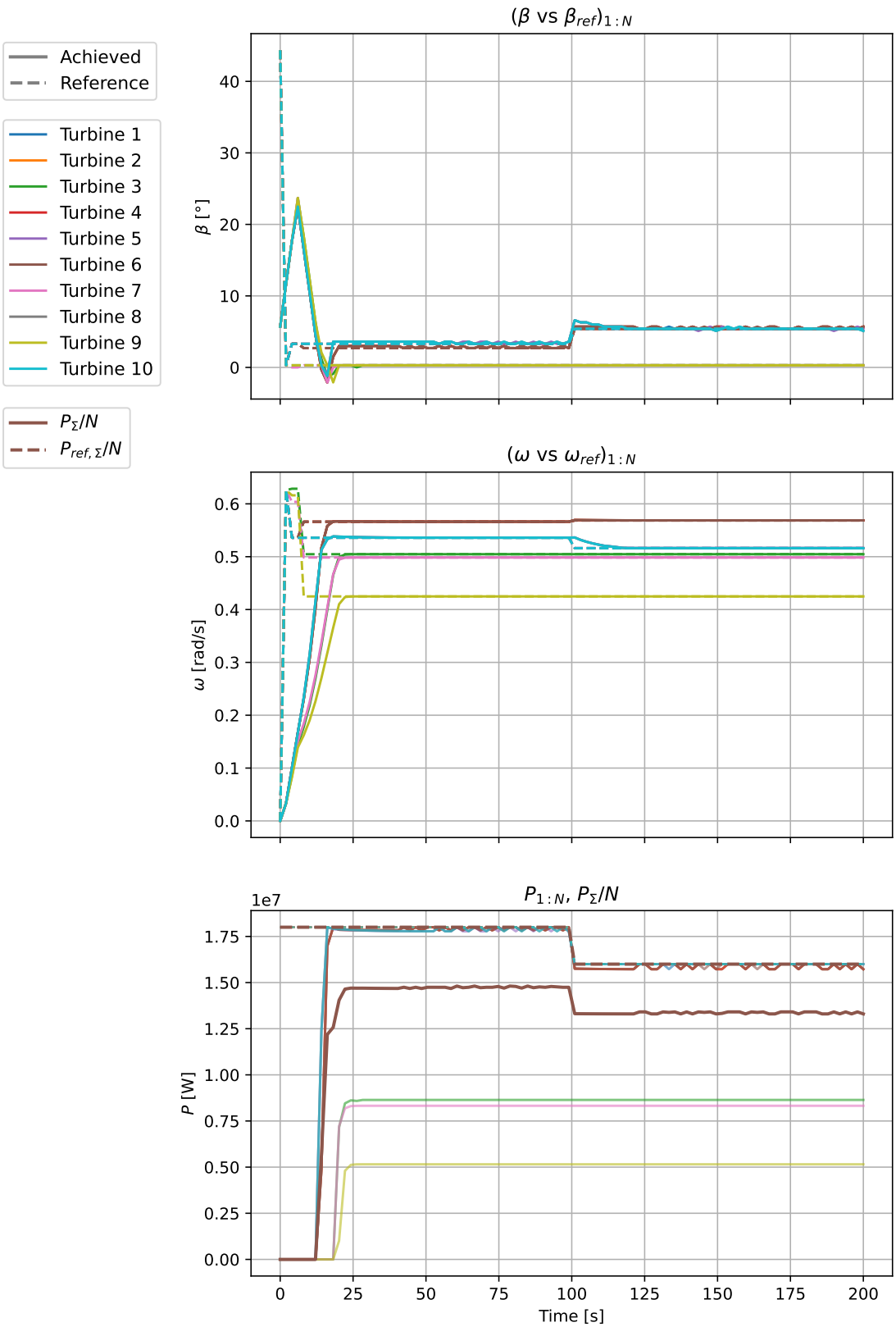


Figure 13: Remotely Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 0°, 0.06)$

7 Centralized Control

The centralized controller should make an informed decision on how to distribute the power demand and it should include yaw steering to increase farm efficiently, especially in critical wind directions where the downstream turbines are in line with the wind. Thus the centralized controller passes down reference power and yaw to each wind turbine. In order to monitor consistency in the data the turbines communicate their states, wind conditions and power output, for all of these quantities it is assumed that full information is available. The communication is shown on Figure 14.

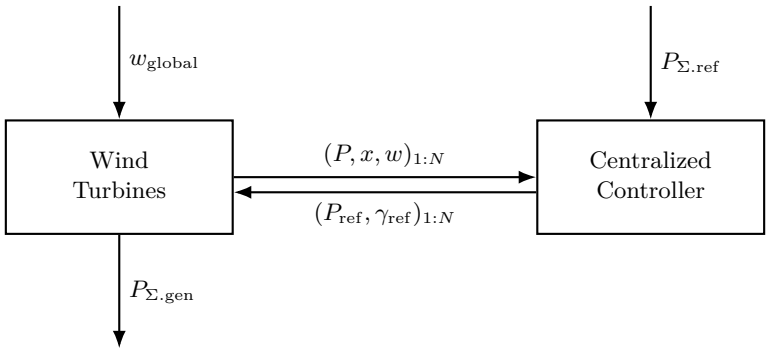


Figure 14: Plant to Centralized Controller Connection with global inputs and output

To construct the centralized controller it is divided into two parts: first the power reference distribution is discussed with zero relative reference yaw in subsection 7.1, then the yaw reference determination is added to it in subsection 7.2.

7.1 Power Distribution

To make a robust controller that can handle the changes in the wind conditions the main information used for the reference power distribution is available power of each turbine given by Equation 29.

$$P_a^{(i)} = \frac{1}{2}\,\rho\,A\,c_{p,\max}\left(V^{(i)}\right)^3 \cos^3\left(\gamma_{rel}^{(i)}\right) I^{(i)}\mu \tag{29}$$

Since the available power can drop in-between P_{ref} signals a safety factor of α is introduced. Meaning if possible each turbine will try to operate below $(1-\alpha)P_a^{(i)}$. If this is not possible the margin equally distributed among the turbines, and if the requested power is more than the available then each turbine operates with the greedy approach. These three scenarios are distinguished and the three control strategies are mapped to them in Equation 30. The set of turbines is expressed as $\mathcal{I}$

$$\kappa = \begin{cases} \kappa_1 & \textit{if } P_{\Sigma.ref} < (1-\alpha)\sum_{i\in\mathcal{I}} P_a^{(i)} \\ \kappa_2 & \textit{else if } P_{\Sigma.ref} < \sum_{i\in\mathcal{I}} P_a^{(i)} \\ \kappa_3 & \textit{else, meaning } P_{\Sigma.ref} \geq \sum_{i\in\mathcal{I}} P_a^{(i)} \end{cases} \tag{30}$$

Control Law: κ_1

$$P_{ref}^{(i)} = min\left(P_{max}, (1-\alpha)P_a^{(i)}\right) \forall i \in \mathcal{I} \tag{31}$$

with, P_{max} defined by the total power output

$$\sum_{j\in\mathcal{I}} P_{ref}^{(j)} = P_{\Sigma.ref} \tag{32}$$

Control Law: κ_2

The second approach equally distributed the margin as defined in Equation 33

$$P_{ref}^{(i)} = P_a^{(i)} + \frac{P_{\Sigma.ref} - \sum_{j\in\mathcal{I}} P_a^{(j)}}{N} \forall i \in \mathcal{I} \tag{33}$$

Control Law: κ_3

The greedy approach, thus each turbine tries to generate as much power as possible, meaning that Equation 34 gives the reference powers.

$$P_{ref}^{(i)} = \inf \forall i \in \mathcal{I} \tag{34}$$

The implementation of V_{cut-in} and $V_{cut-out}$ is not done yet.

7.2 Yaw Steering

The the optimal yaw angles are found using wake simulation and numeric methods. For each combination of wind direction and speed the optimum is found and is stored in a look-up table. Yaw steering is only beneficial when the wakes are strong, therefore the optimal yaw angles are only computed for the greedy approach and a convex combination with zero relative yaw is advised as a future improvement.

$$\gamma_{ref}^{(i)} = \gamma_{rel.steer}(w)^{(i)} + \gamma_{wind}^{(i)} \tag{35}$$

To find the optimal yaw angles for the greedy case Equation 36 two approaches are used. First, the yaw angles are computed in sequence in order of upstream to downstream, with the not computed ones initialized at zero relative yaw. This approach enables to look for the optimal yaw for only one turbine at the time which decreases the number of wake-filed computation from N_m^N to $N_m N$, where N_m is the mash-size of the space of yaw angles per turbine. Secondly, dynamic mashing is used. Per turbine, once the best yaw angle is found the new mash is created around that point. This process repeats until there is significant amount of improvement. The implementation can be found in the Appendix.

$$\gamma_{rel.steer}(w)_{1:N} = \arg\max_{\gamma_{rel.1:N}} P_{\Sigma.gen}(w_{global}, (\gamma_{rel} + \gamma_{wind})_{1:N}) \tag{36}$$

The maximum power that can be obtained this way is visualized on Figure 15 with a global mash size of 300 for the wind direction and 10 for the wind speed. Note that the power is both under and over bounded by V_{cut-in} and $V_{cut-out}$.

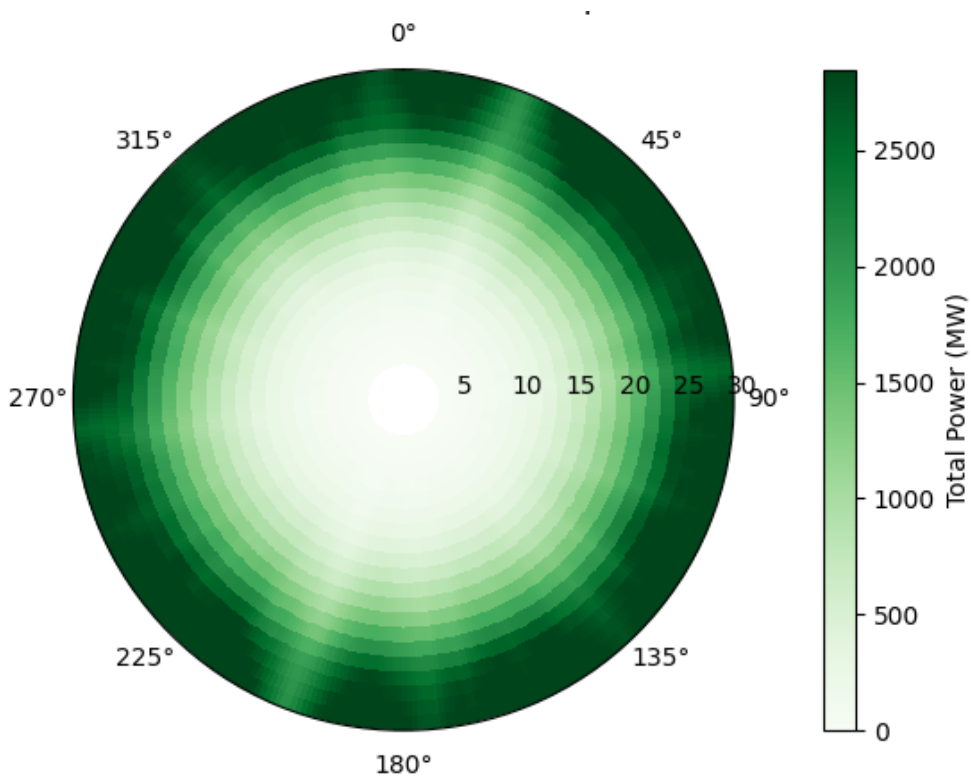


Figure 15: Optimal Total Power Heat-map

One sample for wind speed of $10m/s$ is displayed on Figure 16. It shows that the relative yaw is contained within $[-30°, +30°]$.

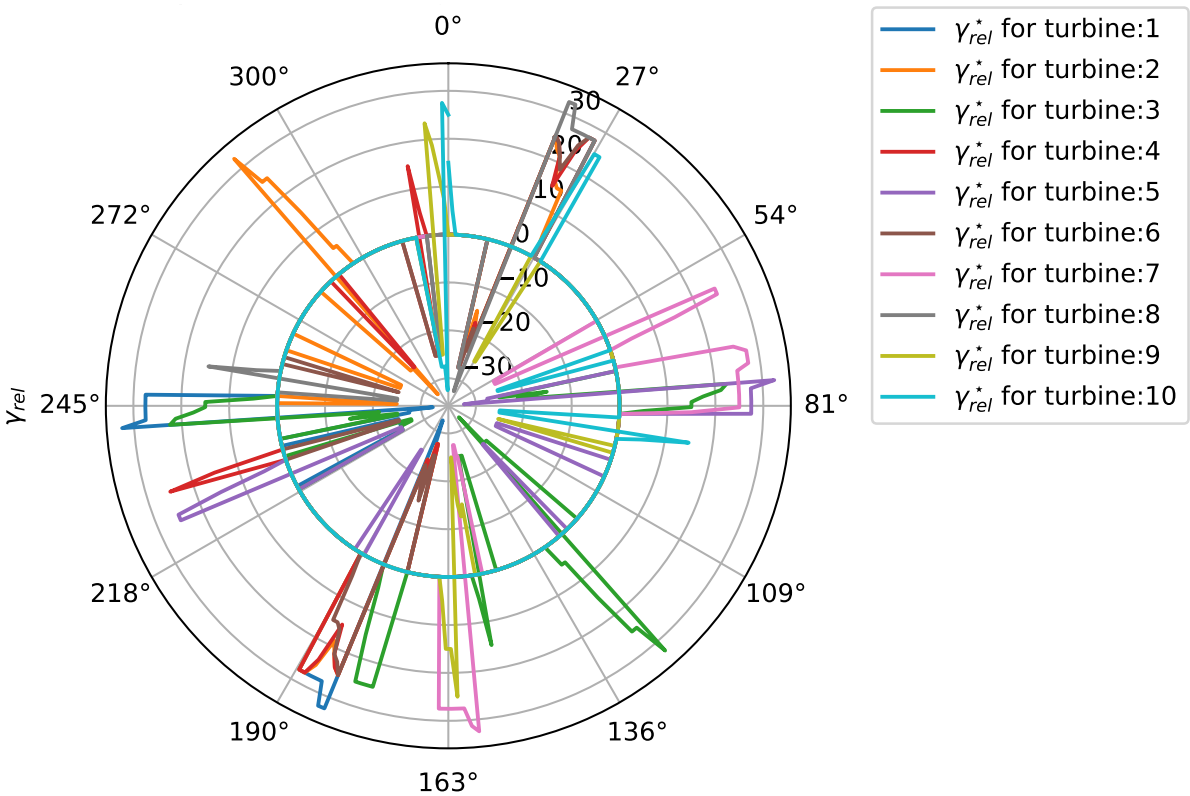


Figure 16: Optimal yaw angles at $V = 10$ [m/s]

7.3 Results

Since the centralized controller was planned to be fully implemented on the PLC, and the PLC communication couldn't not be established, the implementation of the centralized controller is done on the same simulink model as the turbine models and local controllers on the Real-Time Simulator. It is implemented as a MATLAB function block that concatenates its result with the incoming flow-field information, which is a requirement to use the for each subsystem block. This MATLAB function block can be exported as Structured Text (SL) to run it on the PLC. The plots are given for simulations with Speedgoat.

Figure 17 shows only using distributed power tracking, which displayed a significant improvement with respect to Figure 13, where the power reference is evenly split between turbines, even though Figure 17 is at 200°, where more turbines experience wakes. Adding yaw steering to it improves the performance but not to a very high degree. To show a case where yaw steering works well Figure 19 is displayed, where the yaw steering manages to evenly distribute the power between turbines thus resulting in a very high performance. Finally Figure 20 shows the performance under changing wind direction, and shows an overall quite stable performance.

Power Distribution at $200°$

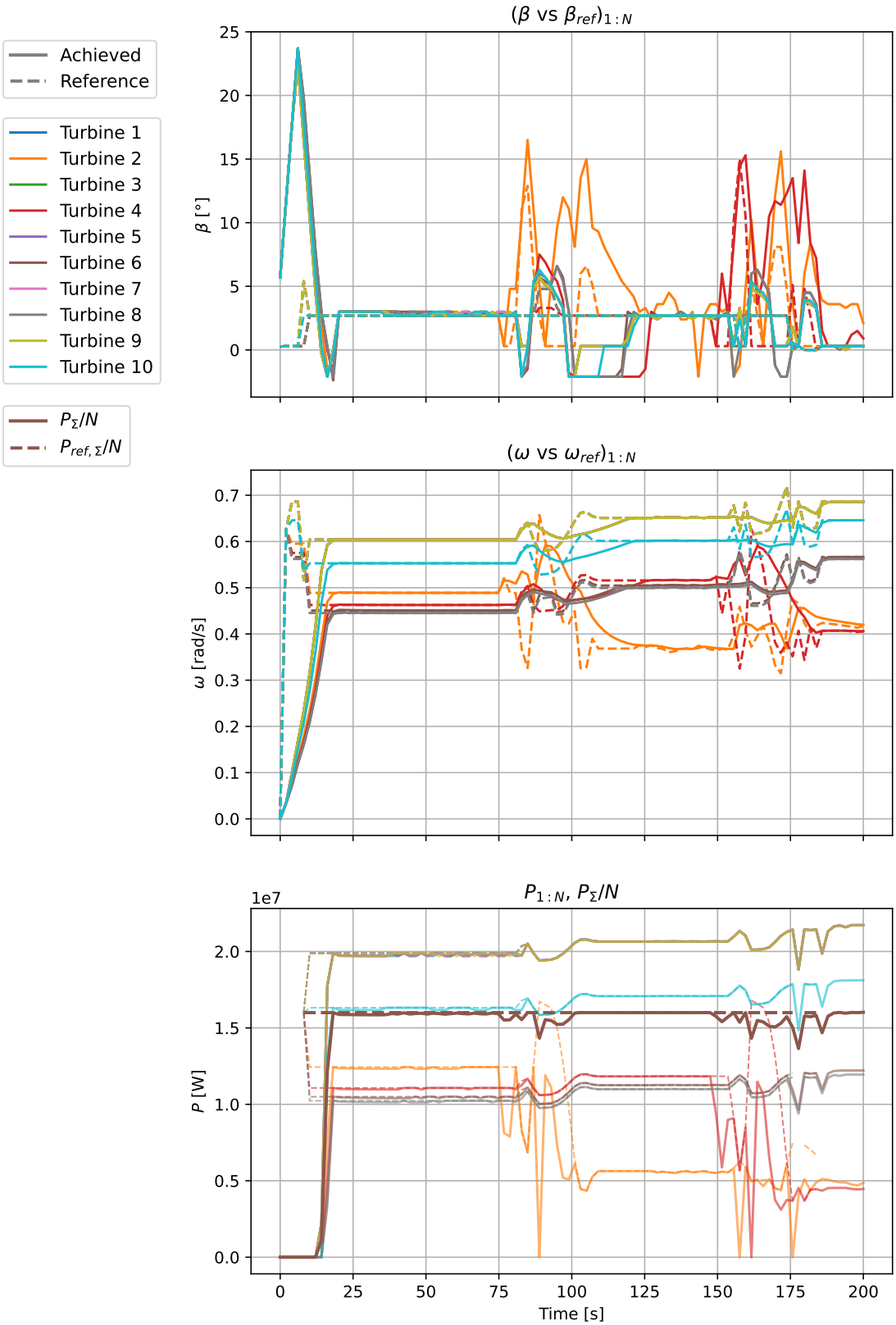


Figure 17: Remotely Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 200°, 0.06)$ power distribution

Power Distribution with Yaw Steering at $200°$

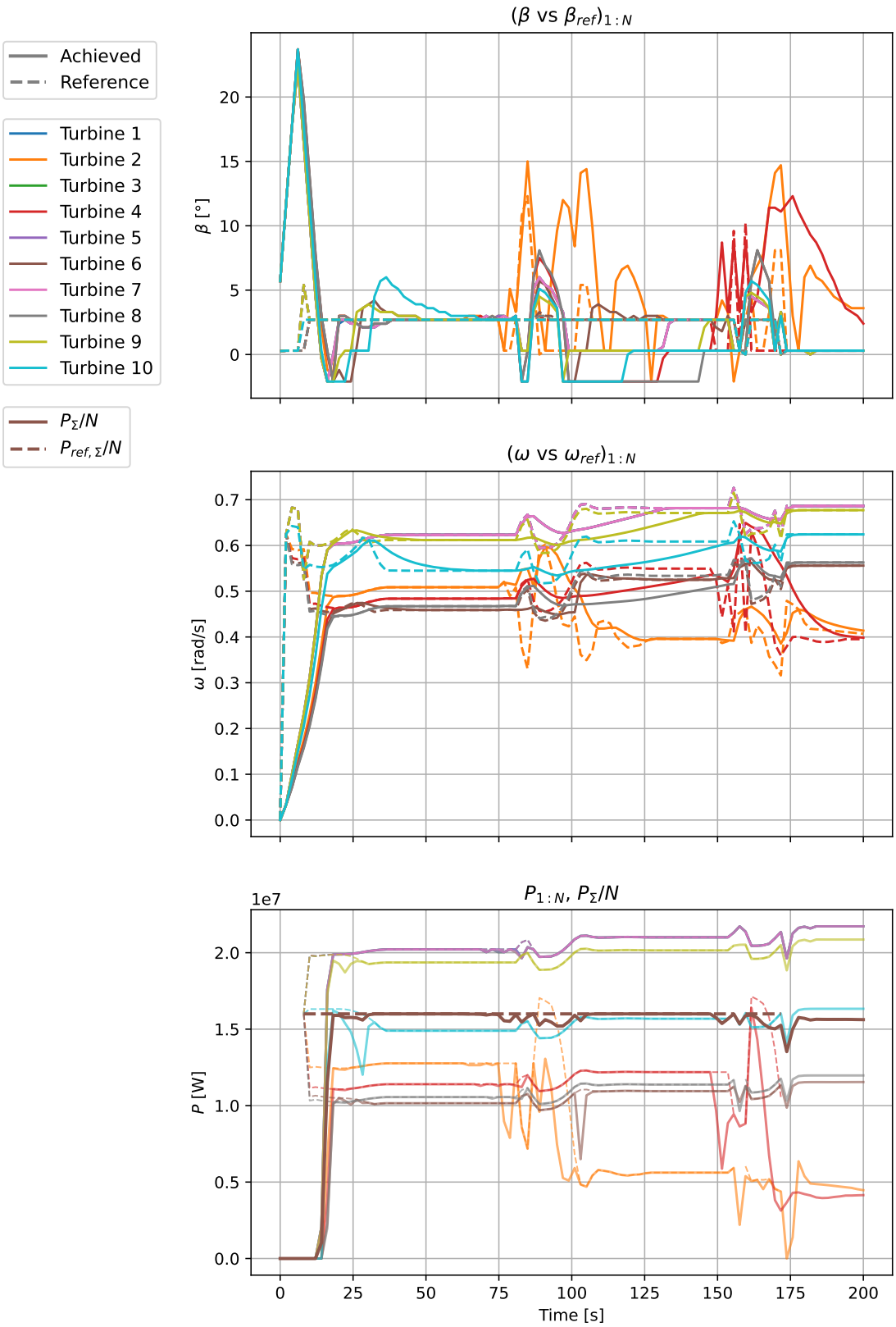


Figure 18: Remotely Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 200°, 0.06)$ with yaw steering and power distribution

Power Distribution with Yaw Steering at $185°$

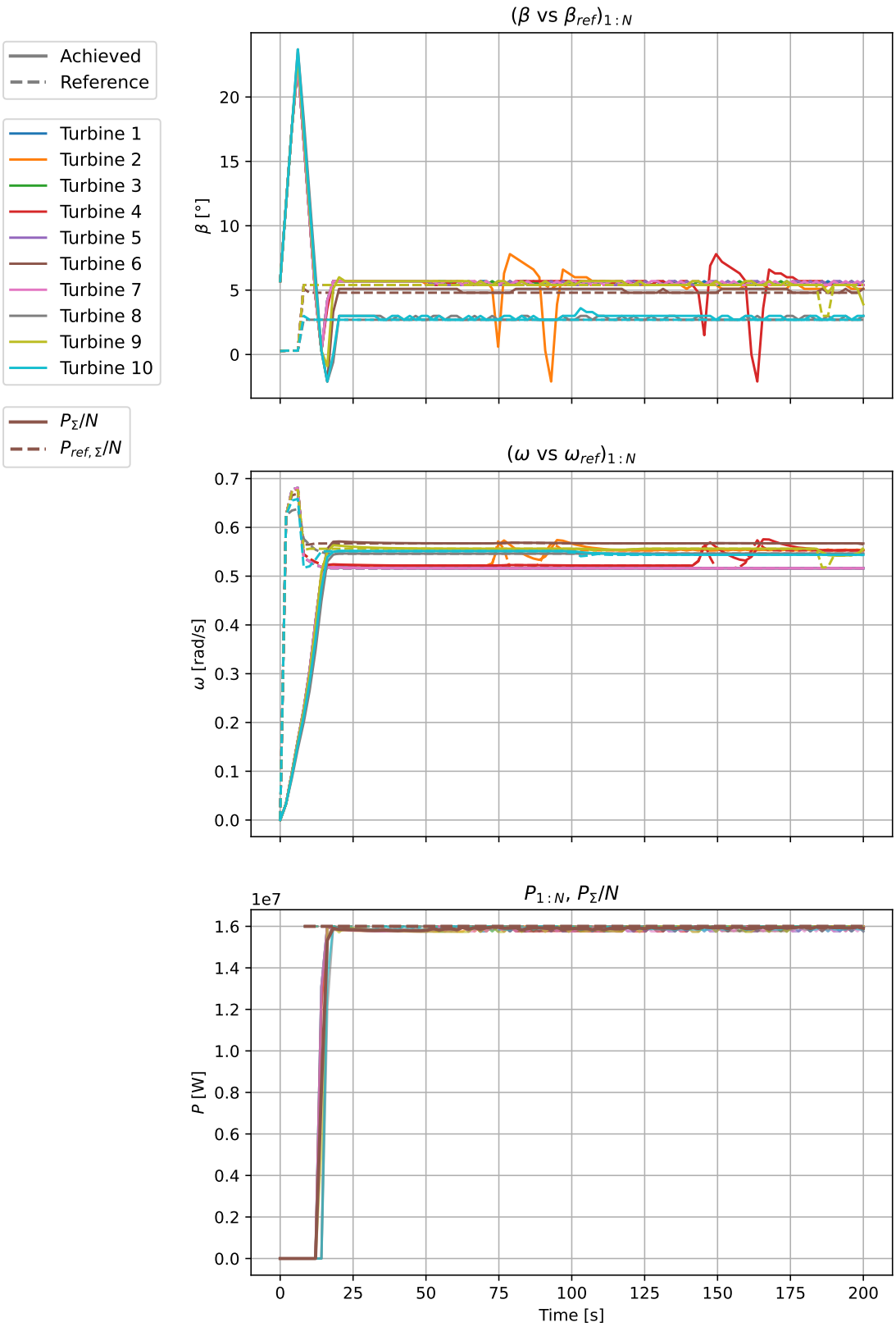


Figure 19: Remotely Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 185°, 0.06)$ with yaw steering and power distribution

Power Distribution with Yaw Steering at interpolated wind between 180° and 280°

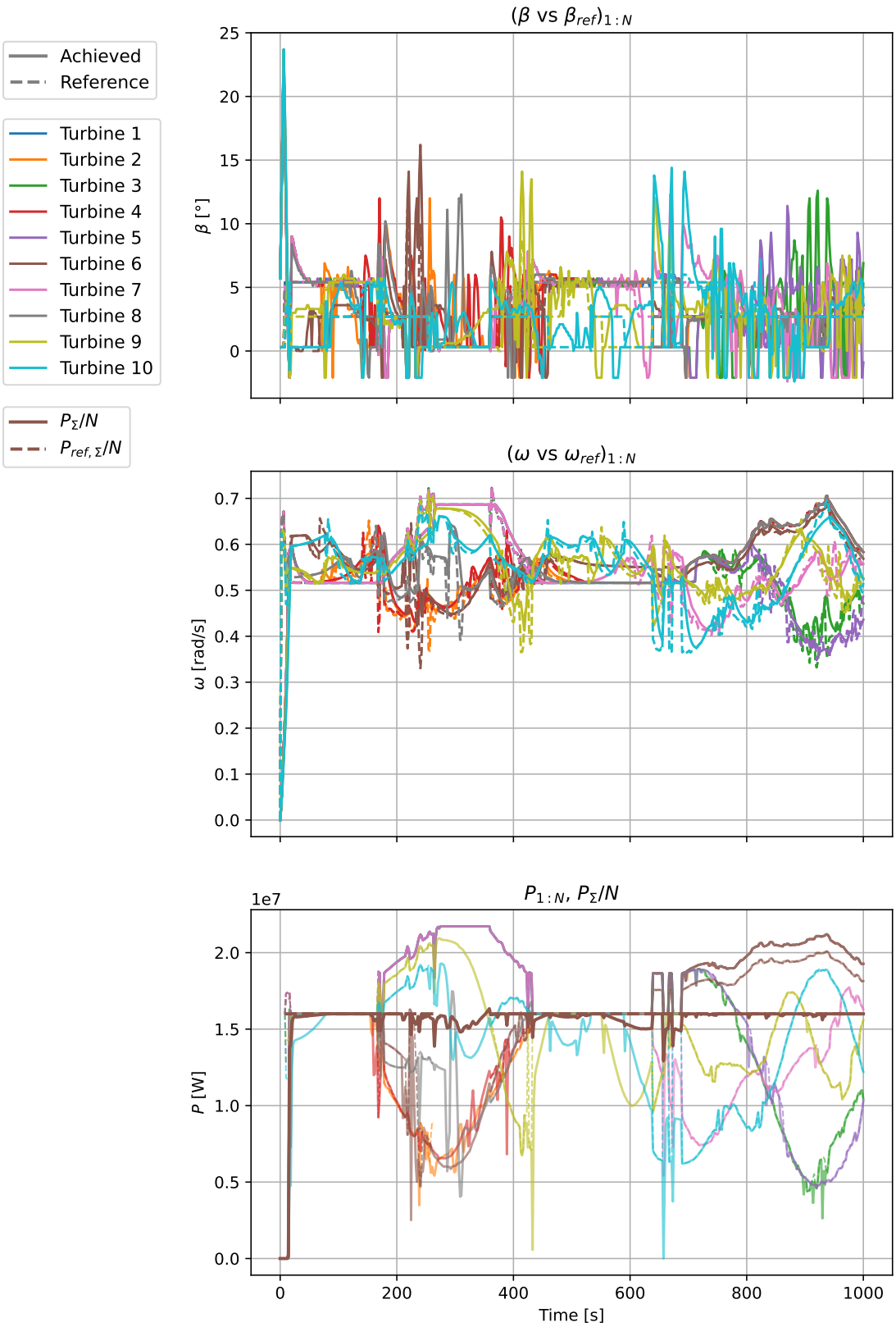


Figure 20: Remotely Run Farm with $(V, \gamma_{wind}, I)_{global} = (11[m/s], 180° - 280°, 0.06)$ with yaw steering and power distribution

8 Implementation on Hardware

8.1 Intended Architecture

The overview of the architecture is shown on Figure 22 . Note that the notation $_i$ denotes that the property is of turbine i, while $_{1:N}$ denotes the stacking of the properties of the turbines from turbine 1 to turbine N. Furthermore the communication is summarized in Table 8.

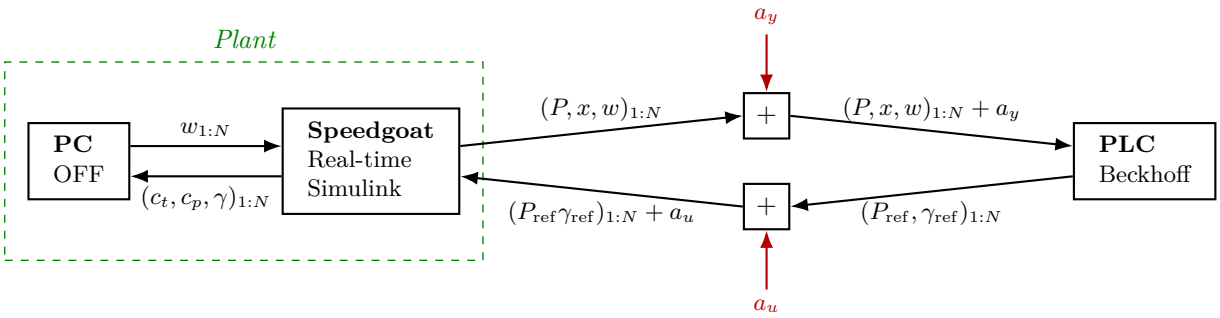


Figure 21: Simulation High-level Architecture with arrowheads

Computational Part	Software	Hardware	Inputs	Outputs
Wake field simulation	OFF	PC	$(c_t, c_p, \gamma)_{1:N}$	$w_{1:N}$
Turbine model & controller	Simulink	Speedgoat	$(w, P_{\text{ref}}, \gamma_{\text{ref}})_{1:N}$	$(P, x, w, c_t, c_p, \gamma)_{1:N}$
Centralized controller	Structured Text	PLC	$(P, x, w)_{1:N}$	$(P_{\text{ref}}, \gamma_{\text{ref}})_{1:N}$

Table 8: Inputs and outputs for each component in the wind farm testbed.

The plant consists of the turbines, and the flow field. The turbines are modelled by a Real Time Simulator (RTS). The flow field determines the wind conditions for each turbine and is computed by OFF, running on a PC. This approach creates a real-time simulation while allowing for flexibility to incorporate a larger variety of wake-field models.

In the plant the wake field is generated with respect to the turbine states, and the turbine dynamics depend on the local wind conditions, this is modelled with the (User Diagram Protocol) UDP communication between the RTS and PC.

The controller runs on the PLC and communicates with the RTS to send power and yaw reference values. The centralized controller receives the power generated, the local wind values (direction, speed, turbulence intensity), and states for each turbine. Ultimately this implementation didn't work, this is further detailed in subsection 8.6.

8.2 Implemented Architecture

As using the PLC is not possible at the moment, the following architecture is implemented, where both centralized controller, localized controllers and turbine models are running on one Speedgoat. With the current set-up it is proven that one Speedgoat can run 40 and it breaks at 50 turbines at $10Hz$.

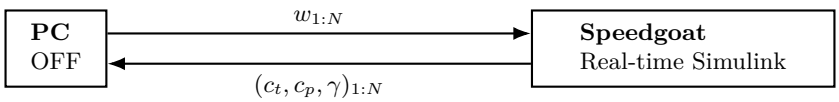


Figure 22: Simulation High-level Architecture

8.3 Using Speedgoat

Assuming that the physical connection between the Speedgoat and the host-PC is active it only takes a few steps to run Speedgoat. Speedgoat runs a simulink model and is accessible via a real-time instance of a simulink model. It has the additional real-time item in the menu bar where the simulink real-time

explorer can be launched to view the connected Speedgoat machine. Then the Speedgoat has to be detected and connected to the simulink model by detect option under the real-time menu item. Once connected the simulink model can be run on Speedgoat by pressing the run on target button.

To connect Speedgoat to the PC or to the PLC with a UDP connection the correct UDP send and receive blocks have to be used. Under real-time Ethernet in the library one can find the TCP and UDP send and receive blocks. Opposite to the standard UDP block in simulink, the UDP receive initiates at a zeros array and uses zero-order-hold (ZOH) on the output of this block so that the real time computations are not stopped.

8.4 RTS - PC Connection

In order to establish the UDP communication the simulink side has no additional logic implemented to read packets. On the OFF side the following logic is used:

- OFF is waiting for for Speedgoat output
- once it receives a packet and there are no more packets in queue it proceeds to run one step of flow field simulation.
- it sends the updated flow-field information

The packet by OFF is sent earlier then the real time the flow field corresponds to, but it is only read when Simulink in Speedgoat listens, which is aligned with real time, therefore the flow information is obtained by Speedgoat at the right time. The communication is further visualized on

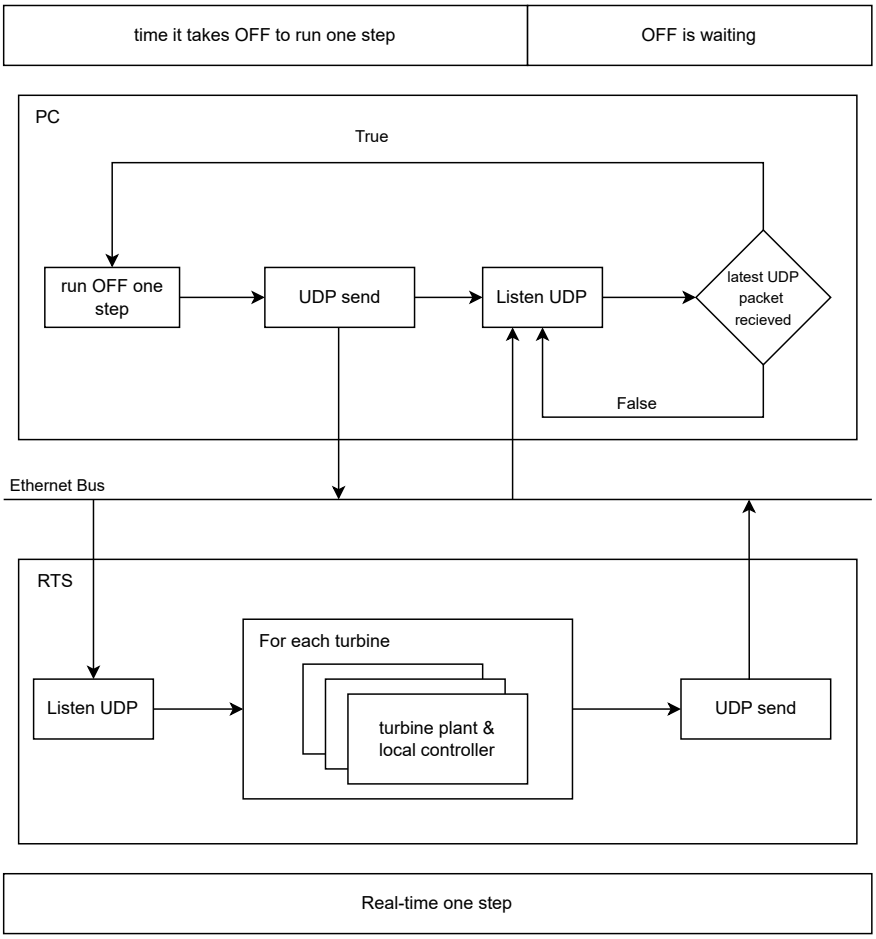


Figure 23: RTS-PC Communication

8.5 Configuring the PLC

The Beckhoff PLC has two types of software associated with it: XAE for configuration and XAR for running the production code on the PLC. There are two main ways to access the PLC. One of them is running the windows operating directly on the PLC itself to configure the network such as changing IP addresses or network interface chard (NIC) attributes. In addition to control desk and other standard windows apps Beckhoff has a local interface to access network attributes. This can also be used to test non Beckhoff software related functions such as testing the network by sending UDP packets with the powershell.

In addition to that Beckhoff has additional software that runs from the host PC such as TwinCAT XAE Shell and TwinCAT package manager. These programs are the primary interfaces to write the code that runs on the PLC and to configure the PLC specific attributes. The following steps need to be executed to set up the PLC to communicate with UDP.

1. Create XAE project with TwinCAT XAE Shell
2. Inside the XAE project Add new PLC project
3. Ensure that the package `TF6310` is installed via TwinCAT package manager and the license `TF6310` is acquired (also included in TF6500, TF6510, TF8810) via XAE project/license/license manager.
4. Add `Tc2_Tcp_Ip` to References under the PLC project.
5. Install the XAR TCP/IP library on the PLC machine itself (starts the TCP/IP server at runtime). This might not be required with a valid license.
6. Add Structured Text (ST) to create the connection using `Tc2_Tcp_Ip`'s function blocks such as create connectionless socket and UDP send to.

Note the following non-intuitive traits of licensing:

1. `Tc2_Tcp_Ip` requires more than the basic license: not only `TC1200`, but also `TF6310`. `TF6311` could also be used, which provides a real-time TCP/UDP implementation using EtherCAT.
2. TwinCAT Package Manager allows installing packages without a valid license.
3. the References section under the PLC project allows adding and compiling the library without license errors, even though no valid license is present.

8.6 Current Issues

The issues are visualized with an option tree on Figure 24 and is further detailed in the follwoing points.

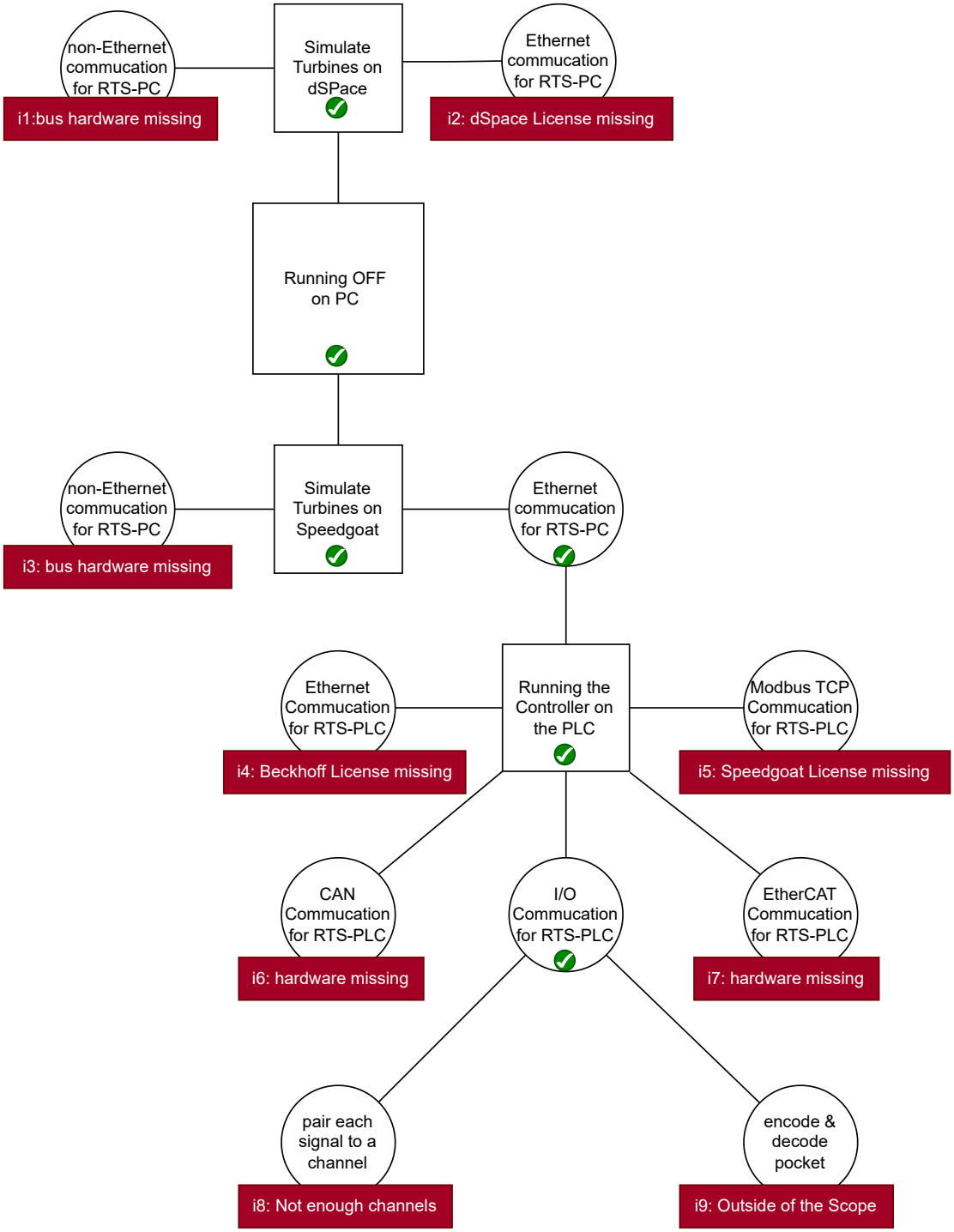


Figure 24: Implementation Option Tree

Each issue has an identifier, e.g. i1 denotes issue 1. each of them are detailed in the following sections:

Issue 1 & Issue 3

The RTS and the PC are connected with an Ethernet connection only. The Ethernet connection serves as the channel to configure the RTS, and Ethernet communication is supported for simulation updates, meaning there are UDP/TCP implementations ready to use for dSpace and Speedgoat, therefore Ethernet connection is the right choice for this communication. It is not advised to build another bus for this communication.

Issue 2

The dSpace hardware available is the MicroAutoBox II model, which requires it's own Ethernet license, which is not available for this project, and support for it's purchase expires at the end of 2026.

Issue 4

`TF6310` licence is missing for PLC to use UDP send and read. It can be purchased to fix the issue.

Issue 5

Since the PLC has Modbus TCP license it was explored as an option, but Speedgoat doesn't have a license to support it.

Issue 6

PLC needs a dedicated hardware interface to work with CAN, which the lab doesn't have.

Issue 7

Both PLC and Speedgoat support EtherCAT but both of them can only function as the master, while in a EtherCAT network there can be only one master. To solve this problem either a microchip chard needs to be added to the Speedgoat that supports functioning as a slave or a bridge can be used that allows both RTS and PLC to send master signals and the bridge converts one of the signals to a slave signal. In both cases there is hardware missing.

Issue 8

Speedgoat only has 4 output analogue channels while the number of values to send is at least 7 per turbine, and for a wind farm to make sense it should include at least 10 turbines, meaning that there are 70 values to send. The problem is also present in the PLC to RTS direction.

Issue 9

I/O channels can be used to make a decoding and encoding of the signals, which is outside of the scope of this project as it is very close to hardware and most likely comes with another set of problems and frameworks to solve on a lower level.

9 User Manual

Even though the implementation is not completed the results can be built on to build the text-bed when the licensing issues are resolved. The project is available at github/arondomotorkohlheb/off_testbed, where `Centralized Controller` contains the yaw steering optimization, `OFF` contains the modified OFF framework, `Speedgoat` contains the simulink models to run on the Speedgoat (MATLAB 2024b), `localTurbines` contain the simulink models to run on the host PC (MATLAB 2025b), `floris` contains the modified floris library, and `collected_data` contains the visualization scripts and data used in this report. The current code support Windows.

9.1 Quick Start

To run the test-bed with the current settings on the local PC two models have to be run around the same time: OFF and the Simulink model.First in order to run OFF the correct environment needs to be set up. `flow_sim_env.yaml` contains the libraries to use but installing floris will include most of the libraries required for the script. In addition to that, the floris folder of this git needs to replace the floris library under the site-packages folder of the environment.

Run on Local PC

- First run localTurbines/turbines_centralized.slx
- After the simulink model started running but before the socket times out: Run OFF/03_Code/off2local.py

The flow of information is displayed by display blocks in the simulink model, and the data is saved under visual_data_pc.npy. The data can be quickly displayed with the use of collected_data/vis_logging_farm.py

Run on PC & Speedgoat

- First start running: Run OFF/03_Code/off2speedgoat.py
- After OFF started running but before the socket times out run Speedgoat/speedgoat_turbines_centralized.slx, for this it might be needed to connect to Speedgoat via detect target and connect to target under the real time menu bar.

The flow of information is displayed by prints in the python script, and the data is saved under visual_data_speedgoat.npy. The data can be quickly displayed with the use of collected_data/vis_logging_farm.py

9.2 Customizing the Wind Farm and Wind Conditions

The setup of the wind farm is described by an `OFF\02_Examples_Cases/03_Cases/**.yaml` file, currently the `windfarm_information_2x5.yaml` file. The configuration file can be changed under line 36: `file_name` in `off2local` and `off2Speedgoat`. This file is set up as an OFF configuration file, where:

1. `sim.sim` describes the step size and duration of the simulation. Since OFF needs to keep up with the real-time simulation and OFF doesn't become slower or faster by changing the step size, there is a lower bound on how small the step size can be. It is found that on Desk3 in the lab having a sample time of $2s$ runs with a comfortable safety margin but lower sample times involve a risk of OFF not keeping up.
2. `farm.layout x/y/z` describe positions of the turbines.
3. `turbine type` describes the type of each turbine, which is set to mask as the turbine modelling happens outside of OFF in the test-bed, except for the diameter which the wake generation uses (floris).
4. `turbine`, `debug`, and `vis` are not used in this testbed.

5. `ambient` configures the wind speed and direction as either constant or linear interpolation between given time points.

Furthermore, the changes here have to be coherent with the contents of `localTurbines/turbine_model_constants.m` and `Speedgoat/turbine_model_constants.m`, which include the geometric properties of the turbines such as

1. A cross sectional area of a turbine
2. Nt number of turbines
3. Ts_inner,Ts_outer,T are the inner (local controller) and outer (centralized controller) sampling times and simulation horizon
4. $x0$ is the initial state, assumed to be the same for each turbine. Having an relative yaw outside of $[-\pi, \pi]$ is not supported by floris, therefore this condition also needs to be checked

The wind turbine is further modelled with c_t and c_p tables read from localTurbines/IEA-22-280-RWT_Cp_Ct_Cq.txt. To change the turbine model this file needs to be replaced with the same layout of: β vector, λ vector, c_p table, c_t table, c_τ (unused) table. When the simulink model is run the tables are extracted each time. Furthermore updating these tables will cause a change in the x_{ref} mapping. For this run gen_xref.m, which generates the xref_optimal file.

1. Replace or update `localTurbines/IEA-22-280-RWT_Cp_Ct_Cq.txt` with the correct layout containing the β vector, λ vector, c_p table, c_t table, and c_τ table.
2. Regenerate the reference mapping by running `gen_xref.m` to produce the updated `xref_optimal` file.

9.3 Customizing Communication

PC

On the PC side the first lines of the main function in both off2local and off2Speedgoat contains the ports and IP addresses to configure. To enable traffic the network should be configured under Open Control Panel → Network and Internet → Network and Sharing Center. The socket time outs can be configured by either changing the argument of the `settimeout()` method or changing the exit conditions on the communication while loops.The decoding of the UDP pocket needs to be aligned with the encoding in simulink. There are multiple options here but the current set up is made such that off2Speedgoat decodes real time UDP pockets while off2local decodes regular UDP pockets.

Speedgoat

To configure the IP addresses of Speedgoat the real time simulink model and the real time explorer apps can be used. In the simulink model the (real-time or regular) UDP send and receive blocks need to be configured for IP address port and pocket size. The real-time UDP blocks use uint8 (pocket size = number of values times 8) encoding while the regular simulink UDP blocks use big-endian (pocket size = number of values times 32).

PLC

To configure the IP address of the PLC sign into the windows running on the PLC and look for the default configuration UI that it automaticity starts. For the different options of network different libraries need to be used. For UDP communication the Tc2_Tcp_Ip library can be used.

9.4 Recommended Solutions to the Standing Problem

Using Another Speedgoat as the centralized Controller

This solution excludes the usage of PLCs, which is not the desired functioning of the test-bed, but it would certainly work and it would raise the accuracy of the simulation and provide a the desired

hardware-in-the-loop, therefore it can be used as a quick solution with no extra costs. In this case the centralized controller block in simulink can be simply moved to another real-time simulink model, and the same real-time UDP blocks can be used for communication as the currently used ones to communicate with the PC, of course with changed settings.

UDP for PLC-RTS communication

A new bus is established to allow this communication to work without any additional traffic. Furthermore the bus has been tested with sending UDP pockets from outside of TwinCAT thus it certainly works, including the reception of pockets by Speedgoat. This solution might not be ideal for testing, as EtherCAT or CAN are more commonly used in industry, but this solution would be the least time consuming because other than the licence all other parts have been tested. Acquiring `TF6310` would enable the test-bed.

Using EtherCAT

EtherCAT would be ideal to make a test-bed that closely resembles industrial applications. To use EtherCAT a hardware interface, a bridge, needs to be implemented to resolve the conflict of both PLC and Speedgoat running as a master. It is highly likely that having the bridge would enable the set-up to communicate, but the experience of this project is not sufficient to guarantee it. The downside of this path is that it probably will bring in new problems to solve, slowing down the process. An additional downside if this solution is the scalability, as including additional computational units, for example another Speedgoat or a PLC, would also require the addition of a new bridge for each new connection.

9.5 Not Recommanded Solutions

Using dSpace

It is not recommended to buy the license for dSpace to enable UDP communication as it wouldn't resolve the issue with the PLC-RTS communication and the current dSpace unit, MicroAutoBox II, will not be supported past 2026.

Using I/O communication

Although it can certainly be done it would shift the focus of the test-bed from cybersecurity to developing an encoding and decoding protocol and would consume a significant amount of time, especially when considering up to 40 turbines, which is the capacity of one Speedgoat at the moment. It would also be challenging to include multiple PLCs or RTSs with I/O communication.

A Changes in OFF

A.1 off_interface updates

- Minor syntax changes due to updated Python version:
  - np.float_ → np.float64
- **off_interface** changes:
  - OFF_path is split with "//" for Windows compatibility
  - `init_simulation_by_path` updated to define the directory of output files
  - __dir_init__:
    - Printing that Floris is installed is commented out
  - _run_yaml_to_dict:
    - `controllersettings`: settings_ctr = None from configuration file
    - Turbine type hardcoded: HAWT_ADM → TurbineMas
    - Removed index input for methods `get_cp` and `get_ct` in TurbineStatesMask class

A.2 Turbine module changes

- Added `TurbineMask` as turbine class
- Added `TurbineStatesMask` as a shell class forwarding turbine state data
- Removed `get_current_cp` and `_ct` methods (including abstract methods), now part of turbine model

A.3 OFF interface and wake model changes

- Turbine generation now masked with turbine states
- Commented out warning for no controller, added `self.controller = None`
- Reduced the following functions to exclude power and ct calculations:
  - get_measurements
  - get_wind_speeds_rp
  - get_measurements_it
- `rgw fmodel.set` now:
  - Sets wind directions, wind speeds, turbulence intensities for all turbines
  - Turbulence intensities from TurbineStatesMask of turbines[] property
- Wake solution decoupled from ct and power computation:
  - Only turbulence intensity and effective velocity computed
  - Custom methods via: off interface → off → wake solver → wake model
  - `fmodel.run()` solves only wakes and velocities (excluding ct, axial induction, power)
  - Constant Floris parameters set at start; dynamic parameters updated once per timestep
- Yaw, ct, cp received from Simulink; axial induction updated by:

$$ai = \frac{1 - \sqrt{1 - c_t}}{2} \tag{37}$$

  via `update_states` method of TurbineStatesMask
- Interpolated states call modified: from [0,1,0,1] → [0,1,1,0] to use the most recent state
- Initialization of all states changed to zero vector

B Changes in Floris

- `floris_model.set()` reinitializes wind farm object
- Inputs: wind, yaw, layout, ct
- Previously, ct computed by a separate model inside Floris; now passed as input

C x_{ref} Mapping

The following logic computes the vectors λ_{ref} and β_{ref} associated with a $c_{p.ref}$ `in x_ref.m`

- Load the aerodynamic lookup tables for the turbine.
  - Extract the $c_p(\lambda, \beta)$ table.
  - Extract the $c_t(\lambda, \beta)$ table.
  - Extract the β grid vector.
  - Extract the λ grid vector.
- Initialize storage containers for:
  - optimal solutions (λ^*, β^*),
  - corresponding c_t^*,
  - reference c_p values.
- Discretize the admissible power coefficient range:
  - Generate $N = 100000$ target values

$$c_p^{ref} \in [0, c_{p,\max}]$$

- For each target c_p^{ref}:
  - Set current best thrust coefficient:

$$c_t^{best} \leftarrow \infty$$

- Sweep the entire (λ, β) grid.
  - For every rectangular tile defined by:

$$(\lambda_i, \lambda_{i+1}) \times (\beta_j, \beta_{j+1})$$

- Evaluate c_p at the four tile corners:
  - (λ_i, β_j),
  - (λ_i, β_{j+1}),
  - (λ_{i+1}, β_j),
  - $(\lambda_{i+1}, \beta_{j+1})$.
- For each of the four tile edges:
  - Check whether c_p^{ref} lies between the two endpoint values.
  - If yes:
    - Linearly interpolate to find $(\lambda_{sol}, \beta_{sol})$.
    - Evaluate:

$$c_t^{sol} = c_t(\lambda_{sol}, \beta_{sol})$$

- Compare candidate solution:
  - If $c_t^{sol} < c_t^{best}$:

    * Update $(\lambda^*, \beta^*) \leftarrow (\lambda_{sol}, \beta_{sol})$.
    * Update $c_t^{best} \leftarrow c_t^{sol}$.

- After scanning all tiles:
  - Store the optimal solution for this c_p^{ref}.
- Repeat for all c_p^{ref} values.
- Save the resulting lookup mapping:
  - $c_p^{ref} \mapsto (\lambda^*, \beta^*, c_t^*)$.
  - Export data for visualization and control reference generation.

D Yaw Optimization with PyWake

The following script computes optimal yaw angles for a wind farm using PyWake and a Gaussian wake model with deflection and turbulence models. The main steps are documented below.

- **Imports:** Essential Python libraries and PyWake modules are imported:
  - Wind turbine and site classes: `WindTurbine`, `UniformSite`, `WindTurbines`.
  - Wake deficit and deflection models: `BastankhahGaussian`, `JimenezWakeDeflection`.
  - Turbulence models: `STF2017TurbulenceModel`.
  - Power vs thrust coefficient function: `PowerCtTabular`.
  - Supporting utilities for farm setup: `build_farm_setup`, `load_windfarm_yaml`, `rotate_points`.
  - Plotting and numerical utilities: `matplotlib.pyplot`, `numpy`.
- **Wind Farm Setup:**
  - Load turbine layout from YAML file.
  - Build farm setup including rotor diameter, hub height, number of turbines, and coordinates.
  - Define a uniform site with mean wind speed ws, turbulence intensity ti, and variable wind direction wd.
- **Turbine Power-Coefficient Model:**
  - Define discrete arrays of wind speeds and corresponding power outputs and thrust coefficients.
  - Create `PowerCtTabular` function to map $V^{(i)} \mapsto (P, C_T)$.
  - Initialize `WindTurbines` class with hub height, diameter, and power/Ct function.
- **Turbine Ordering:**
  - Compute wind-aligned distance for each turbine using layout coordinates and wind direction.
  - Sort turbines in the direction of the wind to determine the optimal evaluation order for yaw optimization.
- **Wake Farm Model Initialization:**
  - Use `BastankhahGaussian` deficit model with `JimenezWakeDeflection` for wake steering.
  - Apply `STF2017TurbulenceModel` for turbulence intensity calculation.
- **Yaw Optimization Loop:**
  - Initialize yaw angles to zero.
  - Iterate over turbines in wind-aligned order:
    * For each turbine, simulate the farm with current yaw configuration.
    * Compute total farm power and store in a dictionary indexed by yaw angle.
    * Iteratively test yaw angles in a specified range $[-20°, 20°]$ to maximize total power.

    - Narrow the search range around the best yaw angle until the relative power improvement is below a threshold (0.01%).
  - Update turbine yaw to the locally optimal value.
  - Plot power vs yaw curve for each turbine.

- **Final Simulation:**
  - Simulate the entire farm with the optimized yaw angles.
  - Compute the total power output
  - Generate wake flow map for visualization and label axes with turbine coordinates.
  - Extract effective turbulence intensity at each turbine for analysis.

- **Outputs:**
  - Optimized yaw angles for each turbine.
  - Total farm power output.
  - Wake map visualization.
  - Turbine-level effective turbulence intensities.

References

Buccafusca, L., & Beck, C. (2021). Multiobjective model predictive control design for wind turbines and farms. *Journal of Renewable and Sustainable Energy*, *13*(3). https://doi.org/10.1063/5.0039707

Zahle, F., Barlas, A., Lønbæk, K., Bortolotti, P., Zalkind, D., Wang, L., Labuschagne, C., Sethuraman, L., & Barter, G. (2024). *Definition of the IEA Wind 22-Megawatt Offshore Reference Wind Turbine* (tech. rep. No. DTU Wind Report E-0243, https://doi.org/10.11581/DTU.00000317). Technical University of Denmark, International Energy Agency.